

Tema 1: Introducción a SQLite

SQLite es un sistema de gestión de bases de datos relacionales que se caracteriza por ser **embebido, sin servidor, autocontenido y de dominio público**. A diferencia de otros sistemas como MySQL o PostgreSQL, SQLite no requiere instalar ni configurar un proceso servidor independiente: toda la base de datos reside en un único archivo en el disco, y las aplicaciones se conectan directamente a él a través de una biblioteca.

¿Por qué SQLite es ideal para nuestro contexto educativo?

El problema que queremos resolver es la dependencia de una conexión a internet para acceder a datos de la PokéAPI. En el aula, los estudiantes pueden trabajar con la API una sola vez, descargar la información, almacenarla localmente y luego consultarla sin red. SQLite encaja perfectamente en este enfoque porque:

- No necesita configuración de servidores ni permisos de administrador.
- Toda la base de datos es un archivo `.db` que se puede copiar, compartir o eliminar fácilmente.
- Está disponible en prácticamente todos los lenguajes y sistemas operativos. En Python viene integrado con el módulo `sqlite3` de la biblioteca estándar.
- Soporta SQL estándar, incluyendo transacciones, claves foráneas, índices, triggers y vistas, lo que permite un aprendizaje completo de bases de datos relacionales.

Características fundamentales

1. Sin configuración de servidor

No hay que instalar un demonio, abrir puertos ni crear usuarios. La base de datos se manipula directamente mediante la biblioteca.

2. Autocontenido y portable

El archivo `.db` es completamente autónomo; puede trasladarse entre sistemas operativos y arquitecturas sin cambios.

3. Tipo de almacenamiento

Todo se guarda en un único archivo de plataforma cruzada. Por ejemplo, `pokemon.db` contendrá todas las tablas, índices, triggers y datos.

4. Transaccional y ACID

SQLite cumple con atomicidad, consistencia, aislamiento y durabilidad, lo que asegura integridad ante caídas o interrupciones.

5. Flexibilidad de tipos

Aunque definamos columnas con tipos (INTEGER, TEXT, REAL, BLOB), SQLite usa un sistema de *afinidad de tipos* en lugar de tipos fijos. Esto significa que podemos almacenar una cadena en una columna declarada como INTEGER si la inserción lo permite, aunque hay recomendaciones para evitar confusiones.

6. Base de datos de uso general

Se utiliza en navegadores, sistemas embebidos, dispositivos móviles (Android, iOS), aplicaciones de escritorio, etc. Es la base de datos más extendida en el mundo.

Limitaciones que debemos conocer

- No fue diseñado para escenarios de alta concurrencia de escritura simultánea (muchos procesos escribiendo a la vez pueden bloquearse). Para nuestra aula, con un único usuario por archivo, no habrá problema.
- No incluye sistema de usuarios y permisos.
- Algunas funcionalidades avanzadas de SQL (como procedimientos almacenados) no están soportadas; hay que implementar la lógica correspondiente en la aplicación o usar alternativas como triggers o funciones definidas por el usuario.

Comparativa con otras bases de datos

Característica	SQLite	MySQL / PostgreSQL
Arquitectura	Embebida (sin servidor)	Cliente-servidor
Instalación	Ninguna, biblioteca incluida	Requiere instalar y configurar
Archivos	Un solo archivo .db	Múltiples archivos y procesos
Concurrencia	Baja (bloqueos a nivel de archivo)	Alta (multiusuario)
Usuarios y permisos	No	Sí
Uso típico	Local, prototipos, móviles	Aplicaciones web y empresariales

Primer contacto práctico: verificar la instalación

Podemos comprobar que SQLite está disponible en nuestro entorno de dos maneras:

- Línea de comandos: abrir una terminal y ejecutar **sqlite3 --version**. Si no está instalado, se puede descargar desde sqlite.org o usar el que viene con Python.
- Desde Python: ejecutar un pequeño script que muestre la versión.

```
import sqlite3
print("Versión de SQLite:", sqlite3.sqlite_version)
```

Si el código anterior no arroja error, ya podemos empezar a trabajar.

Nuestra primera base de datos Pokémon

Para ilustrar el concepto de que **todo está en un archivo**, podemos crear una base de datos desde la terminal y luego inspeccionarla.

```
sqlite3 pokemon.db
```

Dentro del shell interactivo veremos un prompt `sqlite>`. Para salir usamos `.quit`. El archivo `pokemon.db` se creará en el directorio actual si no existía, y ya tendremos nuestra base de datos lista para definir tablas y almacenar información.

En Python, conectar a esa base de datos es igual de sencillo:

```
import sqlite3
conexion = sqlite3.connect("pokemon.db")
# aquí van las operaciones
conexion.close()
```

Rol en el proyecto PokéAPI

La secuencia de trabajo que adoptaremos será:

1. Conectarse a la PokéAPI una vez (requiere internet).
2. Extraer los datos necesarios (nombre, tipos, estadísticas...).
3. Insertar esa información en nuestra base de datos SQLite local.
4. A partir de ese momento, cualquier consulta, reporte o análisis se hará directamente sobre la base de datos local, sin necesidad de internet.

Esto demuestra el valor de SQLite como **solución de persistencia local** y como herramienta para desacoplar la obtención de datos de su consumo. Además, nos permite trabajar los conceptos fundamentales de bases de datos relacionales sin depender de un servicio externo en todo momento.

Tema 2: Instalación y herramientas

Para trabajar con SQLite necesitamos dos componentes básicos: la **biblioteca** que gestiona las bases de datos y una **interfaz** que nos permita enviarle comandos SQL. La biblioteca ya viene incluida en la mayoría de los sistemas operativos modernos y en el intérprete de Python, por lo que en realidad no es necesario instalar nada adicional para comenzar. Sin embargo, conocer las herramientas disponibles nos facilitará enormemente la creación, mantenimiento y depuración de nuestra Pokédex local.

La biblioteca SQLite

SQLite no es un programa independiente, es una librería escrita en C que se integra en la aplicación que la usa. Por ejemplo, cuando en Python escribimos `import sqlite3`, el intérprete carga esa biblioteca y nos ofrece una API para gestionar el archivo de base de datos. Esta biblioteca es la misma que usan navegadores, sistemas operativos, teléfonos móviles y programas de escritorio.

No necesitamos instalar la biblioteca; si tenemos Python 3, ya está disponible. Si trabajamos desde otros lenguajes como Java, C# o Node.js, existen paquetes que la incluyen.

Herramienta de línea de comandos: `sqlite3`

El equipo de SQLite distribuye un programa de terminal llamado `sqlite3` (o `sqlite3.exe` en Windows) que permite interactuar directamente con archivos `.db` mediante un intérprete interactivo. Es ideal para aprender, probar consultas rápidas o automatizar tareas con scripts.

¿Cómo obtenerlo?

- **Linux / macOS:** Es muy probable que ya esté instalado (incluido en las utilidades del sistema). Podemos comprobarlo escribiendo en una terminal:

```
sqlite3 --version
```

Si no está, se instala con el gestor de paquetes habitual (`sudo apt install sqlite3`, `brew install sqlite`, etc.).

- **Windows:** Se descarga un único ejecutable `sqlite3.exe` desde sqlite.org/download.html. Basta con colocar ese archivo en una carpeta que esté en el PATH o ejecutarlo directamente desde su ubicación.
- **Alternativa sin instalación local:** Si no disponemos de permisos, podemos usar la interfaz Python para todo; la terminal es útil pero no imprescindible.

Primeros pasos con la terminal

Al ejecutar `sqlite3` seguido del nombre del archivo (por ejemplo, `sqlite3 pokemon.db`) entramos en un shell interactivo donde podemos escribir comandos SQL y también comandos especiales precedidos por punto. Algunos comandos básicos:

Comando	Descripción
<code>.tables</code>	Lista todas las tablas de la base de datos
<code>.schema</code>	Muestra el esquema (CREATE, índices, triggers)
<code>.exit</code> o <code>.q</code>	Sale del intérprete
<code>.help</code>	Muestra todos los comandos especiales

Podemos ejecutar SQL directamente, por ejemplo:

```
CREATE TABLE prueba (id INTEGER);
INSERT INTO prueba VALUES (1);
SELECT * FROM prueba;
```

Cada instrucción debe terminar con punto y coma. Esta terminal es una herramienta de aprendizaje muy valiosa porque ofrece retroalimentación inmediata y no necesita escribir un programa completo para cada experimento.

El módulo `sqlite3` de Python

Para integrar SQLite en nuestras aplicaciones Python usamos el módulo `sqlite3` de la biblioteca estándar. Como pertenece al lenguaje, no hay que instalar ningún paquete extra: está disponible desde el momento en que instalamos Python (versión 2.5 en adelante, aunque con cambios; en Python 3 funciona sin problemas).

Operaciones elementales con el módulo:

Las acciones siempre siguen la misma secuencia:

1. Conectar a la base de datos con `sqlite3.connect("nombre.db")`. Si el archivo no existe, se crea automáticamente.
2. Obtener un cursor con `conexion.cursor()`.
3. Ejecutar consultas con `cursor.execute()`, `cursor.executescript()`, etc.
4. Confirmar cambios con `conexion.commit()` si hay modificaciones en la base de datos.
5. Cerrar la conexión con `conexion.close()`.

Un mini ejemplo funcional:

```
import sqlite3

# Conexión (crea el archivo si no existe)
conexion = sqlite3.connect("pokemon.db")
cursor = conexion.cursor()

# Crear una tabla sencilla
cursor.execute("""
    CREATE TABLE IF NOT EXISTS tipos (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nombre TEXT UNIQUE NOT NULL
    )
""")

# Insertar un tipo
cursor.execute("INSERT OR IGNORE INTO tipos (nombre) VALUES (?)",
("eléctrico",))

# Confirmar la operación
conexion.commit()

# Consultar
for fila in cursor.execute("SELECT * FROM tipos"):
    print(fila)

conexion.close()
```

Este fragmento ya es perfectamente funcional y no requiere internet. La función `connect()` acepta un nombre de archivo (incluso `:memory:` para bases de datos temporales en RAM). Para nuestros fines,

usaremos siempre un archivo persistente.

Herramientas gráficas (opcional)

Si los estudiantes prefieren una interfaz visual para inspeccionar la base de datos, existen varias aplicaciones de código abierto:

- **DB Browser for SQLite** (<https://sqlitebrowser.org/>): multiplataforma, permite crear, editar, consultar y exportar bases de datos sin escribir código. Muy útil para docentes que quieran mostrar la estructura de las tablas de forma gráfica o para alumnos con poca experiencia en terminal.
- **Extensiones de Visual Studio Code**: hay varias extensiones que permiten abrir archivos `.db`, ver tablas y ejecutar consultas directamente desde el editor.

Aunque no son estrictamente necesarias, estas herramientas pueden facilitar la comprensión del esquema y la resolución de errores durante el desarrollo del proyecto PokéAPI.

Verificación práctica en el entorno educativo

Para asegurarnos de que todo funciona antes de empezar con el volcado de la API, realizamos dos pruebas simples:

1. Prueba con terminal:

Abrimos un shell y creamos una base de datos temporal para ver que `sqlite3` responde:

```
sqlite3 :memory: "SELECT 'SQLite listo';"
```

Si vemos el resultado `SQLite listo`, la terminal opera correctamente.

2. Prueba con Python:

Ejecutamos un script de verificación:

```
import sqlite3
print("Versión:", sqlite3.sqlite_version)
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE test(x)")
print("Conexión y creación OK")
con.close()
```

Si no obtenemos errores, estamos preparados.

Conexión con el proyecto PokéAPI

En los próximos temas utilizaremos exclusivamente Python para todo el flujo de trabajo: descargar información de la API (una sola vez con internet), crear las tablas necesarias, insertar los datos y luego explotarlos. La terminal `sqlite3` nos servirá como soporte para inspeccionar el archivo `pokemon.db`

durante el desarrollo, comprobar que las tablas se crearon correctamente o depurar consultas sin necesidad de escribir un programa completo.

Con estas herramientas instaladas (prácticamente sin instalación), estamos listos para pasar a la creación de nuestra primera base de datos y a la definición de la estructura que albergará los Pokémon.

Tema 3: Crear y conectarse a una base de datos

Una vez que sabemos qué es SQLite y tenemos las herramientas listas, el siguiente paso es **crear físicamente la base de datos** y **conectarnos a ella** desde nuestros programas. En este tema entenderemos cómo funciona el archivo único que almacena toda la información, cómo establecer una conexión desde la terminal y desde Python, y qué configuraciones son indispensables para nuestro proyecto de la Pokédex local. Todo esto se hará sin necesidad de internet, reforzando la idea de que la base de datos reside completamente en nuestro equipo.

El archivo único .db

En SQLite, toda la base de datos (tablas, índices, vistas, datos) se guarda en un solo archivo con extensión **.db**, aunque puede tener cualquier nombre o extensión. No hay catálogos del sistema, demonios en segundo plano ni configuraciones complejas. Ese archivo es autónomo y portable: se puede copiar a otro equipo y funcionará sin migraciones, exportaciones o ajustes adicionales.

Para nuestro proyecto con la PokéAPI, elegiremos el nombre **pokemon.db**. La magia es que cuando ejecutemos la primera instrucción de conexión, si el archivo no existe, SQLite lo crea automáticamente. Si ya existe, simplemente abre la sesión sobre él. No necesitamos ninguna sentencia **CREATE DATABASE** ni permisos especiales de administrador; esto hace que el inicio sea inmediato.

Conexión desde la terminal

La herramienta de línea de comandos **sqlite3** nos permite abrir un archivo .db de forma interactiva. La sintaxis básica es:

```
sqlite3 pokemon.db
```

Al pulsar intro, entraremos en el shell interactivo de SQLite. Si el archivo no existía, puede mostrarse un mensaje como **Connected to a transient in-memory database**, pero en el momento en que ejecutemos una sentencia que requiera escritura (por ejemplo, **CREATE TABLE**), el archivo se materializará en el disco. Una vez dentro del shell, podemos escribir tanto SQL estándar como comandos propios de la herramienta precedidos por un punto.

Algunos comandos de punto útiles:

- **.tables** — lista todas las tablas de la base de datos.
- **.schema** — muestra el esquema completo (CREATE TABLE, índices, triggers).
- **.exit** o **.quit** — salir del intérprete.

- `.help` — lista todos los comandos disponibles.

Esta terminal es muy práctica para inspeccionar rápidamente la base de datos, probar consultas durante el desarrollo o depurar la estructura sin necesidad de escribir un programa en Python. Al cerrar la sesión, los cambios ya están guardados en el archivo (si confirmamos las transacciones; en la terminal, por defecto, cada sentencia se confirma automáticamente salvo que iniciemos una transacción explícita).

Conexión desde Python

El módulo `sqlite3` de la biblioteca estándar de Python nos proporciona la función `connect()`. Esta función recibe la ruta al archivo (o la palabra especial `:memory:`) y devuelve un objeto `Connection` que representa la sesión con la base de datos. El uso más simple es:

```
import sqlite3
conexion = sqlite3.connect("pokemon.db")
```

Si el archivo `pokemon.db` no está en el directorio actual de ejecución del script, se creará allí automáticamente. Se puede especificar una ruta absoluta o relativa. Es importante que la ruta exista en cuanto al directorio contenedor; SQLite no crea directorios, solo el archivo.

Activación obligatoria de claves foráneas

Para que nuestro modelo relacional funcione correctamente, **debemos activar la verificación de integridad referencial**. Por defecto, SQLite no la aplica aunque hayamos declarado `FOREIGN KEY` en las tablas. La instrucción mágica es:

```
conexion.execute("PRAGMA foreign_keys = ON")
```

Este `PRAGMA` se debe ejecutar inmediatamente después de abrir la conexión y antes de cualquier manipulación de datos. En nuestras futuras prácticas lo incluiremos siempre como primer paso tras el `connect()`.

El cursor: nuestra herramienta de ejecución

El objeto `Connection` por sí solo puede ejecutar algunas sentencias (como el `PRAGMA` anterior), pero para lanzar consultas SQL y obtener resultados necesitamos crear un **cursor**. El cursor se obtiene con el método `cursor()`:

```
cursor = conexion.cursor()
```

Una vez tenemos el cursor, podemos usar sus métodos principales:

- `execute(sql, parámetros)` — ejecuta una única sentencia SQL. Admite parámetros para evitar inyección de código.

- `executescript(script)` — ejecuta un script SQL con múltiples sentencias separadas por punto y coma.
- `executemany(sql, lista_de_tuplas)` — ejecuta la misma sentencia para cada tupla de parámetros, ideal para inserciones múltiples.
- `fetchone()`, `fetchall()` — recuperan resultados de una consulta SELECT.

Cuando realizamos cambios en la base de datos (INSERT, UPDATE, DELETE, CREATE), debemos confirmarlos con `conexion.commit()`. Si no los confirmamos, los cambios se perderán al cerrar la conexión (se puede controlar con el modo de transacción, pero la regla general es hacer commit).

Un ejemplo completo de inicio de sesión SQLite con Python sería:

```
import sqlite3

# Crear/conectar a la base de datos
conexion = sqlite3.connect("pokemon.db")

# Activar claves foráneas
conexion.execute("PRAGMA foreign_keys = ON")

# Obtener cursor
cursor = conexion.cursor()

# Aquí irán las operaciones sobre las tablas...

# Confirmar cambios
conexion.commit()

# Cerrar cursor y conexión (el cursor se puede omitir)
conexion.close()
```

Bases de datos temporales en memoria

Para pruebas rápidas, SQLite permite usar la palabra clave `:memory:` en lugar de un nombre de archivo. Esto crea una base de datos exclusivamente en memoria RAM, que desaparece al cerrar la conexión. Es ideal para experimentar sin dejar rastro ni acumular archivos:

```
conexion_temp = sqlite3.connect(":memory:")
conexion_temp.execute("CREATE TABLE prueba(x)")
conexion_temp.close()
```

En nuestras primeras demostraciones en el aula podemos usarla, pero para el proyecto final siempre emplearemos el archivo persistente `pokemon.db` que contendrá los datos extraídos de la API.

Buenas prácticas y cierre ordenado

Siempre que trabajemos con bases de datos:

- Abrir la conexión al principio del script o función.
- Activar `foreign_keys` inmediatamente.
- Crear un cursor (o varios) según necesitemos.
- Confirmar los cambios con `commit()` cuando corresponda.
- Cerrar el cursor y principalmente la conexión al final. Python suele cerrar automáticamente al terminar el script o al salir de un bloque `with`, pero es educado hacerlo explícitamente. La forma más robusta es usar el administrador de contexto:

```
with sqlite3.connect("pokemon.db") as conexion:
    conexion.execute("PRAGMA foreign_keys = ON")
    cursor = conexion.cursor()
    # operaciones...
    conexion.commit()
# Al salir del bloque with, la conexión se cierra automáticamente.
```

Conexión con el proyecto PokéAPI

En nuestro flujo de trabajo, crearemos `pokemon.db` desde cero al iniciar el script que descargue los datos de la API (o en el script de inicialización). Luego, en futuras consultas sin internet, simplemente nos conectaremos a ese mismo archivo, ya lleno de datos. Con este tercer tema, ya estamos listos para empezar a definir la estructura de tablas que albergará la información de la Pokédex, lo cual abordaremos en el siguiente tema.

Tema 4: Tipos de datos y creación de tablas

Ahora que ya sabemos conectarnos a una base de datos SQLite, es el momento de definir la estructura que alojará los datos de nuestra Pokédex local. En este tema aprenderemos los tipos de datos que SQLite ofrece, cómo crear tablas con columnas tipadas, y cómo establecer las restricciones necesarias (claves primarias, unicidad, obligatoriedad) para garantizar la integridad de la información. Todo lo aplicaremos directamente a nuestro dominio Pokémon, sentando las bases del esquema que luego poblaremos con datos de la PokéAPI.

Filosofía de tipos en SQLite

A diferencia de otros sistemas gestores de bases de datos, SQLite no impone un tipado estricto en las columnas. Utiliza un sistema de **afinidad de tipos**: cada columna tiene un tipo recomendado (la afinidad), pero se puede almacenar en ella un valor de cualquier tipo. Las afinidades principales son:

- **TEXT**: texto o cadenas de caracteres.
- **INTEGER**: números enteros.
- **REAL**: números de punto flotante.
- **BLOB**: datos binarios (imágenes, archivos, etc.).
- **NUMERIC**: cualquier valor numérico, tratando de almacenar exactamente el tipo original (entero o real).

Cuando declaramos una columna con un tipo como `VARCHAR(100)`, SQLite no aplica la longitud ni restringe a texto, sino que asigna afinidad TEXT. Lo mismo ocurre con `INT`, `BIGINT`, etc.: acaban teniendo afinidad INTEGER. Esta flexibilidad simplifica el desarrollo pero requiere disciplina para no mezclar datos inapropiados.

Para el proyecto PokéAPI, usaremos los tipos de datos que mejor se adapten a la información real que extraemos: nombres en TEXT, identificadores en INTEGER, alturas y pesos en REAL.

Creación de tablas con CREATE TABLE

La sentencia SQL fundamental para modelar nuestras entidades es `CREATE TABLE`. Su sintaxis básica:

```
CREATE TABLE nombre_tabla (  
    columna1 TIPO RESTRICCIONES,  
    columna2 TIPO RESTRICCIONES,  
    ...  
    RESTRICCIONES_DE_TABLA  
);
```

Las restricciones más habituales que necesitaremos son:

- **PRIMARY KEY**: identifica de forma única cada fila. Puede ser simple o compuesta. Si es un entero con `AUTOINCREMENT`, SQLite generará automáticamente un valor incremental único.
- **NOT NULL**: la columna no admite valores nulos.
- **UNIQUE**: todos los valores de la columna deben ser distintos.
- **DEFAULT** : valor por defecto si no se especifica al insertar.
- **FOREIGN KEY**: establece una relación con otra tabla, garantizando integridad referencial (solo si activamos `PRAGMA foreign_keys = ON`).

Para nuestra Pokédex, crearemos al menos tres tablas:

1. **tipos**: guarda los tipos de Pokémon (fuego, agua, eléctrico...).
2. **pokemon**: almacena los datos básicos de cada Pokémon.
3. **pokemon_tipo**: tabla intermedia para la relación muchos a muchos entre Pokémon y sus tipos.

Diseño de la tabla `tipos`

Comenzamos por los tipos porque son una entidad independiente que solo necesita un nombre y un identificador. La tabla se define así:

```
CREATE TABLE tipos (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    nombre TEXT UNIQUE NOT NULL  
);
```

- `id INTEGER PRIMARY KEY AUTOINCREMENT`: columna que servirá de clave primaria y se autoincrementará con cada inserción. Al ser `INTEGER`, es eficiente y SQLite la mapea internamente al `rowid`.
- `nombre TEXT UNIQUE NOT NULL`: el nombre del tipo (por ejemplo "fire", "water", "electric"). `TEXT` da afinidad de texto, `UNIQUE` impide duplicados, `NOT NULL` exige que siempre se proporcione.

¿Por qué no usar directamente el nombre como clave primaria? Porque las relaciones muchos a muchos funcionan mejor con identificadores numéricos pequeños; además, referenciar un entero es más rápido y ahorra espacio en las tablas intermedias.

Diseño de la tabla `pokemon`

Esta tabla guarda la información principal de cada Pokémon que extraigamos de la API. La PokéAPI nos proporciona campos como `id`, `name`, `height` (en decímetros), `weight` (en hectogramos) y `base_experience`. La tabla se modela así:

```
CREATE TABLE pokemon (
  id INTEGER PRIMARY KEY,
  nombre TEXT NOT NULL,
  altura REAL,
  peso REAL,
  experiencia_base INTEGER
);
```

Observaciones:

- `id INTEGER PRIMARY KEY`: utilizamos el mismo identificador numérico que la PokéAPI asigna a cada Pokémon (Pikachu = 25, Charmander = 4...). Al ser `INTEGER PRIMARY KEY` (sin `AUTOINCREMENT`), lo insertaremos manualmente con el valor exacto de la API. De esta forma mantenemos la coherencia con la fuente original.
- `nombre TEXT NOT NULL`: el nombre del Pokémon en minúsculas (ej: "pikachu"). Obligatorio.
- `altura REAL`: altura en decímetros (un valor como 4 para 0.4 m). Es un número real porque algunas alturas tienen decimales.
- `peso REAL`: peso en hectogramos (un valor como 60 para 6.0 kg).
- `experiencia_base INTEGER`: puntos de experiencia base que otorga al ser derrotado.

Las columnas `altura`, `peso` y `experiencia_base` no llevan `NOT NULL` porque algunos Pokémon pueden no tener valor en la API, aunque en la práctica suelen estar presentes. Dejamos la flexibilidad para no truncar inserciones.

Diseño de la tabla de relación `pokemon_tipo`

Un Pokémon puede tener uno o dos tipos (ejemplo: Charizard es fuego/volador). Un tipo pertenece a muchos Pokémon. Es una relación **muchos a muchos** clásica que resolvemos con una tabla intermedia. Su definición:

```
CREATE TABLE pokemon_tipo (
    pokemon_id INTEGER NOT NULL,
    tipo_id INTEGER NOT NULL,
    PRIMARY KEY (pokemon_id, tipo_id),
    FOREIGN KEY (pokemon_id) REFERENCES pokemon(id) ON DELETE CASCADE,
    FOREIGN KEY (tipo_id) REFERENCES tipos(id) ON DELETE CASCADE
);
```

Explicación detallada:

- `pokemon_id INTEGER NOT NULL` y `tipo_id INTEGER NOT NULL`: cada fila empareja un Pokémon con un tipo. Ambos campos son obligatorios.
- `PRIMARY KEY (pokemon_id, tipo_id)`: la clave primaria compuesta impide que se duplique una misma combinación Pokémon-tipo.
- `FOREIGN KEY (pokemon_id) REFERENCES pokemon(id) ON DELETE CASCADE`: si borramos un Pokémon, automáticamente se eliminan todas sus filas en esta tabla. La cláusula `ON DELETE CASCADE` mantiene la integridad sin intervención del programador.
- `FOREIGN KEY (tipo_id) REFERENCES tipos(id) ON DELETE CASCADE`: lo mismo para los tipos.

Para que estas restricciones de clave foránea actúen, recordemos que en cada conexión debemos ejecutar:

```
conexion.execute("PRAGMA foreign_keys = ON")
```

De lo contrario, las `FOREIGN KEY` se ignorarán, aunque seguirán apareciendo en el esquema.

Restricciones de unicidad y garantía de integridad

Más allá de las claves primarias, hemos utilizado `UNIQUE` en `tipos.nombre` para asegurar que no insertemos dos veces el mismo tipo al importar datos de diferentes Pokémon. Esta restricción es fundamental cuando el script de descarga aplica un `INSERT OR IGNORE` para evitar fallos por duplicados.

Otra restricción habitual es `CHECK`, que permite validar condiciones sobre los valores. Por ejemplo, podríamos añadir:

```
CREATE TABLE pokemon (
    id INTEGER PRIMARY KEY,
    nombre TEXT NOT NULL,
    altura REAL CHECK (altura > 0),
    peso REAL CHECK (peso > 0),
    experiencia_base INTEGER
);
```

Con esto, no se permitirían alturas o pesos negativos. Sin embargo, en nuestro diseño básico lo mantendremos simple e incorporaremos estas comprobaciones cuando sea necesario.

Script completo para crear el esquema desde Python

Uniendo todas las definiciones, podemos confeccionar un script SQL (que grabaremos en un archivo `esquema.sql`) para ejecutarlo de una sola vez desde Python. Dentro del script activamos las claves foráneas y creamos todas las tablas:

```
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS tipos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT UNIQUE NOT NULL
);

CREATE TABLE IF NOT EXISTS pokemon (
    id INTEGER PRIMARY KEY,
    nombre TEXT NOT NULL,
    altura REAL,
    peso REAL,
    experiencia_base INTEGER
);

CREATE TABLE IF NOT EXISTS pokemon_tipo (
    pokemon_id INTEGER NOT NULL,
    tipo_id INTEGER NOT NULL,
    PRIMARY KEY (pokemon_id, tipo_id),
    FOREIGN KEY (pokemon_id) REFERENCES pokemon(id) ON DELETE CASCADE,
    FOREIGN KEY (tipo_id) REFERENCES tipos(id) ON DELETE CASCADE
);
```

Nótese el uso de `IF NOT EXISTS`: útil para no intentar crear tablas que ya existen y evitar errores si se ejecuta el script múltiples veces. En producción se puede controlar mejor, pero para educación es práctico.

Desde Python, la ejecución puede ser directa:

```
import sqlite3

conexion = sqlite3.connect("pokemon.db")
conexion.execute("PRAGMA foreign_keys = ON")
cursor = conexion.cursor()

with open("esquema.sql", "r", encoding="utf-8") as f:
    script = f.read()
cursor.executescript(script)

conexion.commit()
conexion.close()
```

El método `executescript()` puede lanzar múltiples sentencias SQL separadas por punto y coma, exactamente lo que contiene el archivo.

Relación con el proyecto PokéAPI

Con estas tablas ya definidas, tenemos la base estructural para recibir los datos que extraeremos de la API. El mapeo es directo:

- Los tipos de la API se convertirán en filas de la tabla `tipos`; el campo `type.name` del JSON alimentará `nombre`.
- Para cada Pokémon, el campo `id` de la API irá a `pokemon.id`, `name` a `nombre`, `height` a `altura`, `weight` a `peso` y `base_experience` a `experiencia_base`.
- El array `types` del JSON se transformará en una o dos filas en `pokemon_tipo`, vinculando al Pokémon con los tipos correspondientes (los tipos habrán sido previamente insertados en `tipos`).

Esta estructura nos permitirá en futuros temas desarrollar consultas, índices y vistas que aprovechen las relaciones. Ahora ya tenemos el esqueleto de nuestra Pokédex local, completamente funcional y sin necesidad de internet una vez creado.

Tema 5: Modelado de relaciones con el dominio Pokémon

Hasta ahora hemos creado tablas independientes para Pokémon y para tipos. Sin embargo, el verdadero poder de una base de datos relacional reside en cómo conectamos esas tablas entre sí. En este tema modelaremos la relación que existe entre los Pokémon y sus tipos, entenderemos por qué necesitamos una tabla intermedia y diseñaremos cuidadosamente las claves foráneas, la clave primaria compuesta y las reglas de integridad referencial. Todo ello reflejará fielmente la información que extraeremos de la PokéAPI y nos permitirá después recuperarla localmente sin conexión a internet.

Análisis del dominio

De la PokéAPI obtenemos para cada Pokémon un conjunto de datos: su identificador numérico, su nombre, su altura, su peso, su experiencia base y una lista de tipos (por ejemplo, Pikachu es "electric"; Charizard es "fire" y "flying"). De este pequeño universo extraemos dos entidades principales:

- **Pokémon:** representa cada criatura individual.
- **Tipo:** representa una clasificación (fuego, agua, eléctrico, etc.).

La relación natural entre ambas es: *un Pokémon puede poseer uno o dos tipos, y un tipo puede pertenecer a muchos Pokémon*. En términos de bases de datos, esto constituye una **relación muchos a muchos** (N:M).

Por qué no se pueden almacenar los tipos directamente en la tabla Pokémon

Una primera idea podría ser añadir columnas `tipo1` y `tipo2` en la tabla `pokemon`. Esta solución presenta varios problemas:

- Limita artificialmente a dos tipos; aunque actualmente ningún Pokémon tiene tres, el modelo debe ser robusto.
- Dificulta las consultas: para buscar todos los Pokémon de un tipo concreto habría que comprobar dos columnas, y la consulta sería ineficiente.
- Si un tipo cambia de nombre, habría que actualizarlo en muchos lugares.
- Rompe la normalización, generando redundancia.

En su lugar, aplicamos la tercera forma normal y extraemos los tipos a su propia tabla, usando una tabla intermedia para representar la relación.

La tabla intermedia: `pokemon_tipo`

Para plasmar una relación N:M se utiliza una tabla de enlace (también llamada tabla asociativa o tabla de intersección). Esta tabla contiene únicamente las claves foráneas que apuntan a las dos tablas principales y, por lo general, su clave primaria es la combinación de ambas. Nuestra tabla `pokemon_tipo` se define así:

```
CREATE TABLE pokemon_tipo (
  pokemon_id INTEGER NOT NULL,
  tipo_id INTEGER NOT NULL,
  PRIMARY KEY (pokemon_id, tipo_id),
  FOREIGN KEY (pokemon_id) REFERENCES pokemon(id) ON DELETE CASCADE,
  FOREIGN KEY (tipo_id) REFERENCES tipos(id) ON DELETE CASCADE
);
```

Cada fila de esta tabla vincula exactamente un Pokémon con un tipo. Si Charizard (id=6) es fuego y volador, existirán dos filas: (6, id_fuego) y (6, id_volador). A su vez, Pikachu (id=25) tendrá (25, id_electrico). De este modo, cada Pokémon puede tener tantas filas como tipos posea, y cada tipo puede estar presente en tantas filas como Pokémon lo compartan.

Desglose de las restricciones

Las definiciones de las tablas `tipos` y `pokemon` ya fueron presentadas en el tema 4, pero las recordamos para ver cómo encajan:

Tabla `tipos` (lado "uno"):

```
CREATE TABLE tipos (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  nombre TEXT UNIQUE NOT NULL
);
```

Tabla `pokemon` (lado "uno" hacia la intermedia):


```
CREATE TABLE pokemon (  
    id INTEGER PRIMARY KEY,  
    nombre TEXT NOT NULL,  
    altura REAL,  
    peso REAL,  
    experiencia_base INTEGER  
);
```

Sobre `pokemon_tipo`, las restricciones clave son:

- **PRIMARY KEY (pokemon_id, tipo_id)**: clave primaria compuesta por las dos columnas. Esto impide que se registre dos veces la misma combinación Pokémon-tipo. Si se intenta insertar otra vez (25, id_electrico), SQLite lanzará un error de unicidad. Además, internamente crea un índice único que acelera las consultas que filtran por ambas columnas o por el prefijo izquierdo.
- **FOREIGN KEY (pokemon_id) REFERENCES pokemon(id) ON DELETE CASCADE**: indica que `pokemon_id` debe existir en la columna `id` de la tabla `pokemon`. Si se elimina un Pokémon de la tabla `pokemon`, todas sus filas en `pokemon_tipo` se borrarán automáticamente (en cascada). Así no quedan relaciones huérfanas.
- **FOREIGN KEY (tipo_id) REFERENCES tipos(id) ON DELETE CASCADE**: análogo para los tipos. Si borramos un tipo, se eliminan sus ocurrencias en la tabla intermedia (aunque los Pokémon que solo tenían ese tipo no se borrarán; seguirán existiendo en `pokemon`, pero ya sin ese tipo asociado).
- **NOT NULL en ambas columnas**: aseguramos que ningún campo quede vacío; toda relación debe conectar un Pokémon real con un tipo real.

Activación de la integridad referencial

Es crucial recordar que SQLite **no aplica las restricciones FOREIGN KEY por defecto**. Para que todo lo anterior funcione, cada vez que abramos una conexión a la base de datos debemos ejecutar:

```
PRAGMA foreign_keys = ON;
```

En Python, inmediatamente después de `connect()`:

```
conexion.execute("PRAGMA foreign_keys = ON")
```

Si olvidamos este paso, las cláusulas FOREIGN KEY se ignorarán y podremos insertar identificadores sin verificar, dejando relaciones rotas. En nuestro flujo de trabajo con la PokéAPI, esta línea se convierte en obligatoria.

Mapeo desde el JSON de la PokéAPI

Cuando consumamos la API, un Pokémon vendrá representado en JSON de manera similar a esto:

```
{
  "id": 25,
  "name": "pikachu",
  "height": 4,
  "weight": 60,
  "base_experience": 112,
  "types": [
    {"type": {"name": "electric"}}
  ]
}
```

Para Charizard (id=6), el array `types` contendrá dos objetos: `{"type": {"name": "fire"}}` y `{"type": {"name": "flying"}}`. El procedimiento de carga que seguiremos será:

1. Recorrer el array de tipos y, para cada tipo, insertarlo en la tabla `tipos` (si no existe) usando `INSERT OR IGNORE INTO tipos (nombre) VALUES (?)`.
2. Insertar el Pokémon en la tabla `pokemon` con sus datos básicos (id, nombre, altura, peso, experiencia base).
3. Para cada tipo, obtener su `id` desde la tabla `tipos` mediante una consulta por nombre, e insertar un registro en `pokemon_tipo` con los valores (`pokemon_id`, `tipo_id`).

Gracias a las restricciones de clave foránea, si por error intentamos insertar un tipo que no se ha insertado previamente, el gestor rechazará la fila en `pokemon_tipo` y podremos reaccionar. Con el `PRAGMA` activo, la base de datos se protege a sí misma.

Diagrama conceptual del modelo

Para visualizar la estructura sin dibujo, podemos describir las tablas y sus conexiones como texto preformateado. Lo mostramos a continuación en un bloque de código interno para evitar cualquier conflicto con los caracteres especiales:

```
+-----+          +-----+          +-----+
| pokemon |          | pokemon_tipo |          | tipos |
+-----+          +-----+          +-----+
| id (PK) | <-----> | pokemon_id (PK,FK) | id (PK) |
| nombre  |          | tipo_id (PK,FK) | <-----> | nombre |
| altura  |          +-----+          +-----+
| peso    |
| exp_base|
+-----+
```

Las flechas dobles indican la relación muchos a muchos materializada por `pokemon_tipo`. Cada extremo apunta a la clave primaria de la tabla correspondiente.

Consultas que este modelo facilita

Una vez llena la base de datos, podremos realizar consultas como:

- *Obtener todos los tipos de un Pokémon concreto* (unión de las tres tablas filtrando por `pokemon.id`).
- *Contar cuántos Pokémon hay de cada tipo* mediante agrupamiento.
- *Buscar Pokémon que compartan un tipo determinado* (por ejemplo, todos los de fuego).
- *Eliminar un tipo y que automáticamente se desvinculen todos los Pokémon que lo poseían.*

Estas consultas se resolverán con `JOIN` sobre la tabla intermedia, algo que trabajaremos en temas posteriores.

Buenas prácticas con el modelo relacional

- Definir `PRAGMA foreign_keys = ON` al inicio de toda conexión.
- Utilizar `INSERT OR IGNORE` para poblar los tipos sin errores de duplicado.
- Insertar primero las entidades independientes (tipos, Pokémon) y luego las relaciones (`pokemon_tipo`), respetando el orden lógico.
- Al eliminar un Pokémon, confiar en el `CASCADE` para limpiar automáticamente sus relaciones.

Con esto nuestro esquema queda perfectamente modelado para reflejar los datos de la PokéAPI. En el siguiente tema, pasaremos a la inserción masiva de datos desde la API, cerrando así el ciclo de extracción, almacenamiento y desconexión.

Tema 6: Inserción de datos desde la API

Con las tablas ya creadas, el siguiente paso es poblar nuestra base de datos con la información obtenida de la PokéAPI. En este tema aprenderemos cómo trasladar los datos JSON que devuelve la API a filas de nuestras tablas, aplicando estrategias para evitar duplicados, respetar la integridad referencial y usar consultas parametrizadas que prevengan errores e inyecciones SQL. Al finalizar, tendremos una Pokédex local completamente funcional, lista para ser consultada sin internet.

Estrategia general de inserción

Recordemos que nuestro objetivo es desconectarnos de la red una vez descargada la información. Por tanto, la secuencia lógica de trabajo será:

1. Conectarse a la PokéAPI (requiere internet).
2. Para cada Pokémon deseado, extraer un JSON con sus datos.
3. Insertar en la base de datos local los tipos, el Pokémon y las relaciones.
4. Repetir el proceso con todos los Pokémon que queramos almacenar.
5. A partir de ese momento, cualquier consulta se hará sobre la base de datos local.

Al tratarse de una operación que involucra múltiples inserciones relacionadas entre sí, debemos respetar un orden estricto para no violar las claves foráneas:

- **Primero:** insertar los tipos (tabla `tipos`), porque son independientes.
- **Segundo:** insertar el Pokémon (tabla `pokemon`).

- **Tercero:** insertar las filas en la tabla intermedia `pokemon_tipo`, que requiere que tanto el Pokémon como los tipos existan previamente.

Si no seguimos este orden, al intentar referenciar un tipo que aún no ha sido insertado se producirá un error de clave foránea (con el `PRAGMA` activado).

Inserción de tipos: `INSERT OR IGNORE`

La información de tipos viene dentro del JSON de cada Pokémon bajo la clave `types`, que es una lista de objetos. Cada objeto contiene un campo `type.name` con el nombre del tipo (ej. "electric", "fire"). Dado que muchos Pokémon comparten los mismos tipos, al iterar sobre varios Pokémon nos encontraremos repetidamente con los mismos nombres.

Para no interrumpir el proceso con errores de unicidad, utilizamos la cláusula `INSERT OR IGNORE`. Esta sentencia intenta insertar, pero si se viola una restricción (como `UNIQUE` en `tipos.nombre`), simplemente ignora la operación sin lanzar excepción. Combinada con `AUTOINCREMENT`, los tipos duplicados se ignoran y conservamos los IDs originales.

En esencia, para cada tipo encontrado en el JSON ejecutaremos:

```
cursor.execute(
    "INSERT OR IGNORE INTO tipos (nombre) VALUES (?)",
    (nombre_tipo,)
)
```

El uso del parámetro `?` es fundamental: nunca concatenamos directamente el valor en la cadena SQL, porque exponemos la aplicación a inyección de código. El módulo `sqlite3` se encarga de escapar correctamente el valor.

Inserción del Pokémon

Una vez asegurados de que los tipos ya existen (o al menos los hemos intentado insertar), pasamos a la tabla `pokemon`. Aquí usamos un simple `INSERT`, con sus valores extraídos directamente del JSON:

- `id` → de la raíz `id`
- `nombre` → de `name`
- `altura` → de `height`
- `peso` → de `weight`
- `experiencia_base` → de `base_experience`

En Python:

```
cursor.execute(
    "INSERT INTO pokemon (id, nombre, altura, peso, experiencia_base) VALUES"
    "(?, ?, ?, ?, ?)",
    (pokemon_id, nombre, altura, peso, exp_base)
)
```

Como `id` es clave primaria, si intentamos insertar un Pokémon cuyo ID ya existe, obtendremos un error de unicidad. En nuestro diseño asumimos que cada Pokémon se descarga una sola vez, pero si hubiera posibilidad de repetición, podríamos usar `INSERT OR IGNORE` también aquí, o `INSERT OR REPLACE` si quisiéramos actualizar datos.

Inserción de las relaciones: `pokemon_tipo`

Tras insertar el Pokémon, debemos registrar sus tipos en la tabla intermedia. Para cada tipo en el array `types` del JSON:

1. Obtenemos el `tipo_id` correspondiente al nombre. Como los tipos ya están insertados (y si no, `INSERT OR IGNORE` los habrá creado), podemos consultarlos:

```
cursor.execute("SELECT id FROM tipos WHERE nombre = ?", (nombre_tipo,))
fila = cursor.fetchone()
if fila is None:
    # Si por alguna razón no existe, lo insertamos ahora y obtenemos su id
    cursor.execute("INSERT INTO tipos (nombre) VALUES (?)", (nombre_tipo,))
    tipo_id = cursor.lastrowid
else:
    tipo_id = fila[0]
```

2. Insertamos la pareja en `pokemon_tipo`:

```
cursor.execute(
    "INSERT INTO pokemon_tipo (pokemon_id, tipo_id) VALUES (?, ?)",
    (pokemon_id, tipo_id)
)
```

Nótese que al tener una clave primaria compuesta (`pokemon_id, tipo_id`), si por error intentamos insertar la misma relación dos veces, la segunda fallará por unicidad. Podemos también usar `INSERT OR IGNORE` aquí para evitar problemas.

Transacciones y commit

Cada una de estas operaciones de escritura se realiza dentro de una transacción. Por defecto, el módulo `sqlite3` opera en modo de auto-commit cuando se ejecuta una sentencia que modifica la base de datos, pero es buena práctica agrupar las inserciones de un mismo Pokémon en una transacción explícita para mayor eficiencia y consistencia.

Si queremos agrupar muchas inserciones (por ejemplo, al procesar una lista de 151 Pokémon), podemos rodear el bucle con `conexion.commit()` al final, ya que Python inicia una transacción implícita con la primera sentencia de escritura. Alternativamente, podemos usar:

```
with conexion:
    # todas las operaciones
    pass
# al salir del bloque, hace commit automático o rollback si hubo excepción
```

En nuestro caso, simplemente llamaremos a `conexion.commit()` tras cada Pokémon o al finalizar el lote completo. Para simplificar, podemos hacer un único `commit` al final de todas las inserciones, siempre que el script no sea excesivamente largo (en caso de error, se perderían todos los cambios de la transacción, pero como el proceso es lineal, podemos controlarlo).

Ejemplo completo con datos simulados de la API

A continuación mostramos un script en Python que, sin conectarse realmente a internet, simula los datos de Pikachu y Charmander como si los hubiésemos recibido de la PokéAPI. Luego los inserta en nuestra base de datos local. Las funciones de llamada a la API serían análogas usando `requests.get()`, pero aquí nos centramos en la lógica de inserción.

```
import sqlite3

# Datos simulados (normalmente vendrían de requests.get)
pokemones = [
    {
        "id": 25,
        "name": "pikachu",
        "height": 4,
        "weight": 60,
        "base_experience": 112,
        "types": [{"type": {"name": "electric"}}]
    },
    {
        "id": 4,
        "name": "charmander",
        "height": 6,
        "weight": 85,
        "base_experience": 62,
        "types": [{"type": {"name": "fire"}}]
    }
]

# Conexión y configuración
conexion = sqlite3.connect("pokemon.db")
conexion.execute("PRAGMA foreign_keys = ON")
cursor = conexion.cursor()

# Para cada Pokémon
for poke in pokemones:
    poke_id = poke["id"]
    nombre = poke["name"]
    altura = poke["height"]
```

```

peso = poke["weight"]
experiencia = poke["base_experience"]

# 1. Insertar los tipos (primero el tipo, antes del Pokémon)
tipos_nombres = [t["type"]["name"] for t in poke["types"]]
for tipo_nombre in tipos_nombres:
    cursor.execute(
        "INSERT OR IGNORE INTO tipos (nombre) VALUES (?)",
        (tipo_nombre,)
    )

# 2. Insertar el Pokémon
cursor.execute(
    "INSERT INTO pokemon (id, nombre, altura, peso, experiencia_base)
VALUES (?, ?, ?, ?, ?)",
    (poke_id, nombre, altura, peso, experiencia)
)

# 3. Obtener IDs de los tipos e insertar las relaciones
for tipo_nombre in tipos_nombres:
    cursor.execute("SELECT id FROM tipos WHERE nombre = ?", (tipo_nombre,))
    tipo_id = cursor.fetchone()[0]
    cursor.execute(
        "INSERT OR IGNORE INTO pokemon_tipo (pokemon_id, tipo_id) VALUES
(?, ?)",
        (poke_id, tipo_id)
    )

# Confirmar todos los cambios
conexion.commit()
conexion.close()
print("Datos insertados correctamente.")

```

En un escenario real, la lista `pokemones` se obtendría iterando sobre una lista de IDs o URLs, haciendo `requests.get(url).json()` para cada uno. El resto de la lógica de inserción es exactamente la misma.

Control de errores e integridad

Con la activación de claves foráneas, si olvidamos insertar un tipo o el Pokémon, la inserción en `pokemon_tipo` lanzará una excepción `sqlite3.IntegrityError`. Por eso el orden es fundamental.

También debemos manejar posibles excepciones de red al obtener los datos. La base de datos debe permanecer consistente incluso si el proceso se interrumpe a la mitad. Dos enfoques:

- Insertar dentro de una transacción que rodee a cada Pokémon y hacer commit solo si todo el flujo del Pokémon actual es exitoso.
- O bien, usar `INSERT OR IGNORE` en todas las tablas para que el proceso sea idempotente: si se reejecuta el script, los datos existentes se conservan sin duplicados.

Dado que estamos aprendiendo, la segunda opción es más sencilla de entender y aplicar.

Uso de consultas parametrizadas

Ya lo hemos aplicado en los ejemplos, pero merece ser resaltado: **nunca** construiremos una consulta SQL concatenando cadenas con los datos. Siempre usaremos los marcadores `?` y pasaremos una tupla de valores al segundo argumento de `execute()`. Esto evita por completo la inyección SQL, aunque en nuestro contexto de práctica con datos controlados no sea un riesgo real. Es un hábito profesional que se debe adquirir desde el principio.

Inserción masiva con `executemany`

Cuando el número de inserciones es muy grande (por ejemplo, 151 Pokémon con sus tipos), podemos mejorar el rendimiento usando `executemany()`. Por ejemplo, para insertar todas las relaciones de una vez, podríamos construir una lista de tuplas `(pokemon_id, tipo_id)` y ejecutar:

```
datos_relaciones = [
    (25, tipo_electrico_id),
    (4, tipo_fuego_id),
    # ... más
]
cursor.executemany(
    "INSERT OR IGNORE INTO pokemon_tipo (pokemon_id, tipo_id) VALUES (?, ?)",
    datos_relaciones
)
```

Esto reduce el número de llamadas y mejora la velocidad. Lo podemos aplicar tanto a `pokemon` como a `tipos`. Sin embargo, para la práctica inicial, `execute` dentro de un bucle es más didáctico.

Siguientes pasos

Una vez que la base de datos está poblada, nuestra Pokédex local está lista. En los próximos temas, veremos cómo crear índices para acelerar las búsquedas, vistas para simplificar las consultas más comunes, y triggers para automatizar acciones. Pero el núcleo funcional —extraer de la API, guardar localmente, desconectar— ya está completo con lo aprendido hasta aquí.

Con este tema, los estudiantes ya pueden ejecutar el script (con los Pokémon que deseen), cerrar la conexión a internet y luego abrir `pokemon.db` para comprobar que los datos persisten y pueden ser consultados sin red. Ese es el objetivo central del proyecto.

Tema 7: Índices

Cuando nuestra base de datos crece con muchos Pokémon, realizar consultas sin un orden eficiente puede volverse lento. Los índices son estructuras auxiliares que mejoran drásticamente la velocidad de búsqueda, de la misma forma que el índice alfabético de un libro nos evita leer todas sus páginas. En este tema aprenderemos qué son los índices, cómo crearlos en SQLite, en qué casos convienen (y en cuáles

no), y cómo aplicarlos a nuestro proyecto de la Pokédex local para que las consultas sean rápidas incluso cuando hayamos descargado cientos de Pokémon desde la API.

¿Qué es un índice?

Un índice en base de datos es una estructura interna que asocia los valores de una o varias columnas con la ubicación física de las filas correspondientes en la tabla. SQLite implementa los índices mediante árboles B, una estructura equilibrada que permite localizar un valor en tiempo logarítmico en lugar de tener que recorrer todas las filas una a una.

Cuando ejecutamos una consulta con una cláusula **WHERE** sobre una columna indexada, el motor de la base de datos no necesita hacer un barrido secuencial de toda la tabla: consulta el índice, encuentra la posición exacta y accede directamente a los registros. Para bases de datos pequeñas la diferencia no se nota, pero cuando almacenamos cientos o miles de Pokémon, la mejora es sustancial.

Índices automáticos en nuestro esquema

SQLite crea índices automáticamente para ciertas restricciones:

- **Clave primaria:** toda **PRIMARY KEY** genera un índice único. En nuestra tabla **pokemon**, el campo **id** ya está indexado, por lo que buscar por ID de Pokémon es eficiente desde el principio.
- **Restricción UNIQUE:** cada columna con **UNIQUE** también tiene un índice asociado. En la tabla **tipos**, el campo **nombre** tiene un índice único que impide duplicados y acelera las búsquedas por nombre.

Por tanto, cuando hacemos **SELECT id FROM tipos WHERE nombre = 'electric'** para obtener el ID de un tipo antes de insertar una relación, ese acceso ya es rápido gracias al índice único automático.

Sin embargo, otras columnas que usaremos frecuentemente en consultas no disponen de índice, como **pokemon.nombre**. Si queremos buscar Pokémon por nombre, el motor recorrerá todas las filas hasta encontrar coincidencia. Ahí es donde crearemos nuestros propios índices.

Creación de índices con CREATE INDEX

La sentencia SQL para crear un índice es:

```
CREATE INDEX nombre_indice ON tabla (columna1, columna2, ...);
```

Los índices pueden ser sobre una sola columna o sobre varias (índices compuestos). También pueden ser únicos, añadiendo la palabra **UNIQUE**, lo cual impide además valores duplicados.

Para nuestra Pokédex, el caso de uso más evidente es la búsqueda de Pokémon por nombre. Añadimos el siguiente índice:

```
CREATE INDEX idx_pokemon_nombre ON pokemon (nombre);
```

Desde ese momento, cualquier consulta que filtre por `nombre` (o que ordene por él con `ORDER BY nombre`) se beneficiará de ese índice y se ejecutará mucho más rápido.

Podemos agregar este índice justo después de crear las tablas, añadiéndolo a nuestro archivo `esquema.sql`:

```
CREATE INDEX IF NOT EXISTS idx_pokemon_nombre ON pokemon (nombre);
```

El uso de `IF NOT EXISTS` evita errores si el índice ya fue creado en una ejecución anterior.

Índices en la tabla intermedia

La tabla `pokemon_tipo` tiene una clave primaria compuesta (`pokemon_id`, `tipo_id`). SQLite genera automáticamente un índice único para esa combinación. Eso acelera las consultas que buscan por ambos campos a la vez, pero ¿qué ocurre si queremos buscar todos los Pokémon de un tipo concreto? Es decir:

```
SELECT pokemon_id FROM pokemon_tipo WHERE tipo_id = ?;
```

En una clave primaria compuesta, el índice se construye sobre la pareja (`pokemon_id`, `tipo_id`). Si solo filtramos por `tipo_id`, que es la segunda columna del índice, SQLite no puede aprovechar el índice de manera óptima (el índice está ordenado primero por `pokemon_id`). Para que esa consulta frecuente sea eficiente, podemos crear un índice adicional sobre `tipo_id`:

```
CREATE INDEX idx_pokemon_tipo_tipo ON pokemon_tipo (tipo_id);
```

Con este índice, responder a "dame todos los Pokémon de tipo fuego" será inmediato. De igual modo, si a menudo consultamos los tipos de un Pokémon concreto filtrando por `pokemon_id`, el índice de la clave primaria ya lo cubre porque `pokemon_id` es la primera columna del índice compuesto.

Cuándo crear un índice y cuándo no

Crear índices no es gratuito. Cada índice ocupa espacio en el archivo `.db` y, sobre todo, ralentiza las operaciones de escritura (`INSERT`, `UPDATE`, `DELETE`), porque el motor debe actualizar tanto la tabla como todos los índices afectados. En nuestro escenario de Pokédex, las escrituras ocurren una sola vez (al importar desde la API) y las lecturas serán constantes, por lo que el coste de mantenimiento es asumible.

Algunas pautas para decidir:

- Indexar columnas que aparecen frecuentemente en `WHERE`, `JOIN`, `ORDER BY` o `GROUP BY`.
- No indexar columnas con pocos valores distintos (por ejemplo, un campo booleano). La ganancia es mínima.
- No sobrecargar de índices una tabla con muchas inserciones continuas. En nuestro caso, solo escribimos al poblar la base de datos.

Visualización de índices existentes

Desde la terminal de `sqlite3`, podemos ver los índices de la base de datos con el comando `.indices`. Desde Python, podemos consultar la tabla maestra `sqlite_master`:

```
cursor.execute("SELECT name, sql FROM sqlite_master WHERE type='index'")
for fila in cursor.fetchall():
    print(fila)
```

En el shell interactivo:

```
sqlite3 pokemon.db ".indices"
```

Esto nos mostrará los índices automáticos (creados por `PRIMARY KEY` y `UNIQUE`) y los que hayamos añadido manualmente.

Ejemplo práctico: búsqueda con y sin índice

Imaginemos que tenemos 151 Pokémon en la tabla. Sin índice sobre `nombre`, buscar un Pokémon por nombre requiere recorrer en promedio la mitad de la tabla. Con índice, el motor desciende por el árbol B en unas pocas operaciones. Aunque con 151 registros la diferencia de tiempo es imperceptible para un humano, podemos simular un escenario didáctico usando la funcionalidad `EXPLAIN QUERY PLAN` de SQLite, que muestra cómo se ejecutará una consulta:

```
EXPLAIN QUERY PLAN SELECT * FROM pokemon WHERE nombre = 'pikachu';
```

Antes de crear el índice, el plan dirá `SCAN TABLE pokemon` (recorrido secuencial). Después de crear `idx_pokemon_nombre`, dirá `SEARCH TABLE pokemon USING INDEX idx_pokemon_nombre`, confirmando que el índice está siendo utilizado.

Índices compuestos

Si en el futuro necesitásemos consultas que filtren simultáneamente por varias columnas, podríamos crear un índice compuesto. Por ejemplo, si a menudo buscamos Pokémon por tipo y además los ordenamos por experiencia base, un índice (`tipo_id`, `experiencia_base`) sería útil. Se crea así:

```
CREATE INDEX idx_poke_tipo_exp ON pokemon_tipo (tipo_id, pokemon_id);
```

El orden de las columnas en el índice compuesto importa: la primera columna debe ser la que se use en la cláusula `WHERE` con igualdad; las siguientes pueden usarse para ordenación o para otras condiciones.

En nuestro proyecto actual no es necesario, pero conviene conocer la posibilidad.

Incorporación a nuestro script de inicialización

Para que todo quede integrado, nuestro archivo `esquema.sql` debería incluir los índices justo después de crear las tablas. El contenido completo actualizado sería:

```
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS tipos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT UNIQUE NOT NULL
);

CREATE TABLE IF NOT EXISTS pokemon (
    id INTEGER PRIMARY KEY,
    nombre TEXT NOT NULL,
    altura REAL,
    peso REAL,
    experiencia_base INTEGER
);

CREATE TABLE IF NOT EXISTS pokemon_tipo (
    pokemon_id INTEGER NOT NULL,
    tipo_id INTEGER NOT NULL,
    PRIMARY KEY (pokemon_id, tipo_id),
    FOREIGN KEY (pokemon_id) REFERENCES pokemon(id) ON DELETE CASCADE,
    FOREIGN KEY (tipo_id) REFERENCES tipos(id) ON DELETE CASCADE
);

CREATE INDEX IF NOT EXISTS idx_pokemon_nombre ON pokemon (nombre);
CREATE INDEX IF NOT EXISTS idx_pokemon_tipo_tipo ON pokemon_tipo (tipo_id);
```

Al ejecutar este script con `cursor.executescript(script)`, las tablas y los índices se crean en el orden correcto.

Relación con el proyecto de la Pokédex

Una vez que los estudiantes hayan descargado una cantidad considerable de Pokémon desde la API (por ejemplo, los primeros 151), las consultas de la Pokédex local seguirán siendo instantáneas gracias a los índices. Las operaciones más típicas son:

- Buscar un Pokémon por nombre (para mostrar sus datos en pantalla).
- Dado un tipo, listar todos los Pokémon que lo poseen (usando el índice sobre `tipo_id`).
- Contar cuántos Pokémon hay de cada tipo (consulta agregada que se beneficia del índice en la tabla intermedia).

Todas ellas se optimizan con los índices definidos, demostrando así que la base de datos no solo almacena datos, sino que lo hace de manera eficiente para su recuperación.

Práctica propuesta

Para afianzar el concepto, los estudiantes pueden:

1. Llenar la base de datos con muchos Pokémon (más de 100).
2. Ejecutar **EXPLAIN QUERY PLAN** antes y después de crear el índice sobre **nombre**.
3. Medir el tamaño del archivo **pokemon.db** antes y después de añadir los índices, para comprobar el espacio adicional que ocupan.
4. Crear índices adicionales que crean convenientes y justificar su necesidad.

Tema 8: Vistas

A medida que nuestra base de datos crece, ciertas consultas se repiten una y otra vez. Por ejemplo, cada vez que queremos mostrar un Pokémon junto con los nombres de sus tipos, tenemos que escribir un **JOIN** entre tres tablas. Las **vistas** nos permiten guardar esas consultas como si fueran tablas virtuales, simplificando el acceso a los datos y haciendo el código más legible y mantenible. En este tema aprenderemos a crear vistas en SQLite, entenderemos cómo se comportan y construiremos una vista práctica para nuestra Pokédex local.

¿Qué es una vista?

Una vista es una consulta almacenada en la base de datos que se comporta como una tabla virtual. No guarda datos por sí misma; cada vez que la consultamos, SQLite ejecuta internamente la consulta que la define y devuelve los resultados actualizados. Podemos usar las vistas en sentencias **SELECT** exactamente igual que si fueran tablas normales, incluso combinarlas con **WHERE**, **ORDER BY** u otras cláusulas.

Las principales ventajas de las vistas son:

- **Simplicidad:** encapsulan lógica compleja (como múltiples **JOIN** o funciones de agregación) detrás de un nombre sencillo.
- **Reutilización:** una vez creada, cualquier parte de la aplicación puede usarla sin repetir la consulta.
- **Seguridad:** podemos exponer solo ciertas columnas de las tablas originales, ocultando información sensible o no relevante.

Creación de una vista con CREATE VIEW

La sintaxis básica es:

```
CREATE VIEW nombre_vista AS
SELECT columnas
FROM tablas
WHERE condiciones;
```

Se puede anteponer **CREATE TEMP VIEW** para crear una vista temporal que desaparece al cerrar la conexión, pero normalmente usaremos vistas permanentes que queden almacenadas en el archivo **.db**.

En nuestro proyecto, una consulta muy frecuente será "mostrar cada Pokémon con los nombres de sus tipos". Sin una vista, esa consulta implica unir las tablas **pokemon**, **pokemon_tipo** y **tipos**, y además

agrupar por Pokémon para concatenar los tipos con `GROUP_CONCAT()`:

```
SELECT p.id, p.nombre,  
       GROUP_CONCAT(t.nombre, ', ') AS tipos  
FROM pokemon p  
JOIN pokemon_tipo pt ON p.id = pt.pokemon_id  
JOIN tipos t ON pt.tipo_id = t.id  
GROUP BY p.id;
```

Cada vez que necesitemos ese listado, tendríamos que repetir esta consulta. Para evitarlo, encapsulamos todo en una vista:

```
CREATE VIEW vista_pokemon_tipos AS  
SELECT p.id, p.nombre,  
       GROUP_CONCAT(t.nombre, ', ') AS tipos  
FROM pokemon p  
JOIN pokemon_tipo pt ON p.id = pt.pokemon_id  
JOIN tipos t ON pt.tipo_id = t.id  
GROUP BY p.id;
```

A partir de ahora, podemos consultar la vista como si fuera una tabla:

```
SELECT * FROM vista_pokemon_tipos WHERE nombre = 'charmander';
```

Esta sencillez es especialmente útil para los estudiantes cuando tengan que depurar o presentar los datos de su Pokédex en pantalla.

Las vistas son dinámicas

Es muy importante comprender que una vista no almacena los datos resultantes en el momento de su creación. Cada vez que se consulta, la base de datos reevalúa la consulta subyacente. Por tanto:

- Si insertamos nuevos Pokémon o tipos, la vista reflejará esos cambios automáticamente.
- No hay que refrescarla ni actualizarla; siempre está sincronizada con las tablas base.

Esto las diferencia de una "tabla materializada" (que algunos motores soportan), pero para nuestro uso es perfecto porque la información de la Pokédex es estática una vez importada, y si alguna vez añadimos más Pokémon, la vista se actualiza al instante.

Uso de vistas en consultas más complejas

Las vistas pueden combinarse con otras tablas o incluso con otras vistas. Por ejemplo, podríamos crear una segunda vista que, basándose en `vista_pokemon_tipos`, filtre solo los Pokémon de un cierto tipo usando `WHERE tipos LIKE '%fuego%'`. O podríamos cruzar la vista con una hipotética tabla de movimientos.

En nuestro script de Python, consultar la vista es idéntico a consultar una tabla normal:

```
cursor.execute("SELECT * FROM vista_pokemon_tipos")
resultados = cursor.fetchall()
for fila in resultados:
    print(f"ID: {fila[0]}, Nombre: {fila[1]}, Tipos: {fila[2]}")
```

Esto reduce la cantidad de SQL explícito en el código, haciendo que los programas sean más cortos y menos propensos a errores de escritura de consultas.

Limitaciones de las vistas en SQLite

Aunque las vistas son muy útiles, tienen restricciones que debemos conocer:

- **No se pueden indexar directamente.** Los índices deben crearse sobre las tablas base, no sobre las vistas. Por eso, para que las consultas sobre una vista sean rápidas, las tablas subyacentes deben estar bien indexadas. En nuestro caso, los índices sobre `pokemon.nombre`, `pokemon_tipo.pokemon_id` (automático por clave primaria) y `pokemon_tipo.tipo_id` garantizan que las operaciones con `JOIN` dentro de la vista sean eficientes.
- **No admiten `ORDER BY` en su definición** (aunque podemos añadirlo al consultar la vista). Si intentamos incluir `ORDER BY` dentro de la vista, SQLite lo ignorará en la mayoría de contextos; es una buena práctica no incluirlo y dejar que cada consulta decida el orden.
- **Son de solo lectura** si la consulta subyacente contiene `JOIN`, `GROUP BY`, `DISTINCT` o funciones de agregación. No podremos hacer `INSERT`, `UPDATE` o `DELETE` directamente sobre `vista_pokemon_tipos` porque SQLite no puede traducir esas operaciones a las tablas base de forma única. Para modificaciones siempre debemos actuar sobre las tablas reales.

Integración en nuestro esquema

Igual que hicimos con los índices, podemos añadir la creación de la vista a nuestro archivo `esquema.sql`. De esta manera, cada vez que inicialicemos la base de datos, la vista estará disponible:

```
CREATE VIEW IF NOT EXISTS vista_pokemon_tipos AS
SELECT p.id, p.nombre,
       GROUP_CONCAT(t.nombre, ', ') AS tipos
FROM pokemon p
JOIN pokemon_tipo pt ON p.id = pt.pokemon_id
JOIN tipos t ON pt.tipo_id = t.id
GROUP BY p.id;
```

La cláusula `IF NOT EXISTS` evita errores si el script se ejecuta más de una vez.

Eliminar y modificar vistas

Para eliminar una vista usamos `DROP VIEW`:

```
DROP VIEW IF EXISTS vista_pokemon_tipos;
```

Si queremos modificarla, debemos eliminarla y volver a crearla, ya que SQLite no soporta **ALTER VIEW**. En nuestro entorno eso se traduce en actualizar el archivo `esquema.sql` y, si la base de datos ya está en uso, ejecutar el drop y el create manualmente.

Uso didáctico en el proyecto Pokédex

La vista `vista_pokemon_tipos` será probablemente el punto de entrada principal para consultar la información de la Pokédex local una vez desconectados de internet. Los estudiantes pueden:

- Listar todos los Pokémon con sus tipos ordenados alfabéticamente.
- Buscar un Pokémon por nombre y ver sus tipos concatenados.
- Exportar el resultado de la vista a un archivo CSV u otro formato para su análisis.

Todo ello sin necesidad de recordar las uniones SQL. Además, si más adelante se añaden más detalles al modelo (habilidades, movimientos), se pueden crear nuevas vistas que combinen esa información, manteniendo la sencillez en la capa de presentación.

Conclusión

Las vistas son una herramienta de abstracción muy potente que nos ayuda a manejar la complejidad de las consultas relacionales. En nuestro proyecto de la Pokédex, nos permitirán tener una tabla virtual lista para consumir desde Python o desde la terminal, ocultando los **JOIN** y mostrando directamente la información que interesa: el Pokémon y sus tipos. En el siguiente tema abordaremos los triggers, que llevan la automatización un paso más allá.

Tema 9: Triggers (disparadores)

En los temas anteriores hemos escrito toda la lógica de inserción y consulta desde Python. Pero SQLite nos permite definir acciones automáticas que se ejecutan cuando ocurren ciertos eventos sobre las tablas, incluso sin intervención del programa que se conecta. Estos automatismos se llaman **triggers** (disparadores). En este tema aprenderemos qué son, cómo crearlos y cómo utilizarlos para añadir funcionalidades a nuestra Pokédex local, como llevar un registro automático de cada nuevo Pokémon que almacenemos.

¿Qué es un trigger?

Un trigger es un conjunto de instrucciones SQL que se asocian a una tabla y se ejecutan de forma automática en respuesta a un evento determinado: **INSERT**, **UPDATE** o **DELETE**. Los triggers permiten:

- Registrar automáticamente auditorías (logs de cambios).
- Validar o modificar datos antes de que se escriban.
- Mantener coherencia entre tablas cuando las restricciones de clave foránea no cubren ciertos casos.
- Disparar acciones en cascada que no estén definidas a nivel de esquema.

En nuestro proyecto, queremos que cada vez que insertemos un Pokémon en la tabla `pokemon`, quede constancia en una tabla de auditoría, sin necesidad de que el script Python lo haga explícitamente. Esto demuestra que la base de datos puede encargarse de ciertas tareas administrativas incluso cuando trabajamos sin internet.

Sintaxis básica de CREATE TRIGGER

La estructura general es:

```
CREATE TRIGGER nombre_trigger
  [BEFORE|AFTER] [INSERT|UPDATE|DELETE] ON nombre_tabla
  [FOR EACH ROW]
BEGIN
  -- sentencias SQL
END;
```

Algunas notas importantes:

- **Momento:** `BEFORE` (antes de que se realice la operación) o `AFTER` (después de realizada). Para auditoría usaremos `AFTER INSERT`, porque queremos registrar el hecho una vez confirmado que el Pokémon se insertó correctamente.
- **Evento:** `INSERT`, `UPDATE` o `DELETE`. Es posible combinar varios con `INSERT OR UPDATE`, etc.
- **FOR EACH ROW:** SQLite solo admite triggers por fila (row-level). El bloque de código se ejecuta una vez por cada fila afectada.
- **Dentro del bloque:** podemos escribir cualquier sentencia SQL, incluyendo `INSERT`, `UPDATE`, `DELETE`, `SELECT`. No se permite `COMMIT` o `ROLLBACK`.
- **Acceso a valores:** disponemos de las referencias `NEW` (para el nuevo valor en `INSERT/UPDATE`) y `OLD` (para el valor anterior en `UPDATE/DELETE`). Con `INSERT`, `OLD` no tiene sentido.

Ejemplo: tabla de auditoría de nuevos Pokémon

Queremos que cuando nuestro script (o cualquier otro acceso) inserte un Pokémon, se escriba automáticamente un registro en una tabla `log_nuevos` indicando el `pokemon_id` y la fecha. Para ello, primero creamos la tabla de auditoría:

```
CREATE TABLE log_nuevos (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  pokemon_id INTEGER NOT NULL,
  fecha TEXT DEFAULT (datetime('now')),
  FOREIGN KEY (pokemon_id) REFERENCES pokemon(id) ON DELETE CASCADE
);
```

La columna `fecha` toma por defecto la fecha y hora actual usando la función interna `datetime('now')`, que devuelve la fecha en formato UTC. Luego creamos el trigger:

```
CREATE TRIGGER trg_nuevo_pokemon
AFTER INSERT ON pokemon
FOR EACH ROW
BEGIN
    INSERT INTO log_nuevos (pokemon_id) VALUES (NEW.id);
END;
```

Explicación:

- **AFTER INSERT ON pokemon**: se activa justo después de que una fila se haya insertado en **pokemon**.
- **FOR EACH ROW**: el bloque se ejecuta por cada fila insertada (si insertamos de una vez varias, se lanza una vez por fila).
- **NEW.id**: referencia al valor de la columna **id** de la fila que se acaba de insertar. Como es **AFTER**, la fila ya existe y podemos usar su **id**.
- El trigger inserta en **log_nuevos** el **pokemon_id** y deja que el valor por defecto de **fecha** haga el resto.

A partir de ese momento, cualquier inserción en **pokemon** (ya sea desde Python, desde la terminal o desde otro trigger) generará automáticamente un registro en **log_nuevos**. Si se intenta insertar un Pokémon cuyo **id** ya existía y la operación falla por unicidad, el trigger no se ejecuta, porque la inserción no llega a realizarse.

Acceso a OLD y NEW en diferentes eventos

Evento	NEW	OLD
INSERT	Contiene los nuevos valores	NULL
UPDATE	Contiene los nuevos valores	Contiene los valores anteriores
DELETE	NULL	Contiene los valores eliminados

En un trigger **BEFORE** podemos incluso modificar los valores de **NEW** antes de que se graben (por ejemplo, poner el nombre en minúsculas). En nuestro caso no lo necesitamos, pero es una posibilidad.

Triggers para validación de datos

Podríamos crear un trigger que impida insertar un Pokémon con peso negativo. Aunque podríamos usar una restricción **CHECK**, un trigger permite emitir un mensaje de error personalizado mediante la función **RAISE()**. Por ejemplo:

```
CREATE TRIGGER trg_valida_peso
BEFORE INSERT ON pokemon
FOR EACH ROW
BEGIN
    SELECT CASE
        WHEN NEW.peso <= 0 THEN
            RAISE(ABORT, 'El peso debe ser mayor que cero')
```

```
END;  
END;
```

Esto es opcional y no lo incluiremos en el esquema básico, pero ilustra el poder de los triggers para mantener la integridad.

Actualización automática de datos relacionados

Imaginemos que quisiéramos mantener una columna `cantidad_pokemon` en la tabla `tipos` que almacene cuántos Pokémon hay de cada tipo. Podríamos usar triggers para incrementarla o decrementarla al insertar o eliminar en `pokemon_tipo`. Esto se sale del alcance del proyecto, pero muestra cómo los triggers ayudan a desnormalizar sin perder consistencia.

Cómo ver los triggers creados

En la terminal de SQLite, el comando `.schema` muestra todos los triggers. En Python podemos consultar la tabla `sqlite_master`:

```
cursor.execute("SELECT name, sql FROM sqlite_master WHERE type='trigger'")  
for fila in cursor.fetchall():  
    print(fila[0], fila[1])
```

Incorporación al esquema de la Pokédex

Nuestro archivo `esquema.sql` crece. Ahora añadimos la tabla de auditoría y el trigger después de las tablas principales:

```
PRAGMA foreign_keys = ON;  
  
CREATE TABLE IF NOT EXISTS tipos (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    nombre TEXT UNIQUE NOT NULL  
);  
  
CREATE TABLE IF NOT EXISTS pokemon (  
    id INTEGER PRIMARY KEY,  
    nombre TEXT NOT NULL,  
    altura REAL,  
    peso REAL,  
    experiencia_base INTEGER  
);  
  
CREATE TABLE IF NOT EXISTS pokemon_tipo (  
    pokemon_id INTEGER NOT NULL,  
    tipo_id INTEGER NOT NULL,  
    PRIMARY KEY (pokemon_id, tipo_id),  
    FOREIGN KEY (pokemon_id) REFERENCES pokemon(id) ON DELETE CASCADE,
```

```

    FOREIGN KEY (tipo_id) REFERENCES tipos(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS log_nuevos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    pokemon_id INTEGER NOT NULL,
    fecha TEXT DEFAULT (datetime('now')),
    FOREIGN KEY (pokemon_id) REFERENCES pokemon(id) ON DELETE CASCADE
);

CREATE INDEX IF NOT EXISTS idx_pokemon_nombre ON pokemon (nombre);
CREATE INDEX IF NOT EXISTS idx_pokemon_tipo_tipo ON pokemon_tipo (tipo_id);

CREATE VIEW IF NOT EXISTS vista_pokemon_tipos AS
SELECT p.id, p.nombre,
       GROUP_CONCAT(t.nombre, ', ') AS tipos
FROM pokemon p
JOIN pokemon_tipo pt ON p.id = pt.pokemon_id
JOIN tipos t ON pt.tipo_id = t.id
GROUP BY p.id;

DROP TRIGGER IF EXISTS trg_nuevo_pokemon;
CREATE TRIGGER trg_nuevo_pokemon
AFTER INSERT ON pokemon
FOR EACH ROW
BEGIN
    INSERT INTO log_nuevos (pokemon_id) VALUES (NEW.id);
END;

```

Nótese que hemos usado **IF NOT EXISTS** también para el trigger, aunque en SQLite esta sintaxis no es válida para triggers; se requiere un workaround. Normalmente, para evitar errores en ejecuciones repetidas del script, se puede envolver el **CREATE TRIGGER** con un **DROP TRIGGER IF EXISTS** previo, o simplemente no incluirlo en el script de inicialización y ejecutarlo por separado una sola vez. En el ámbito educativo se puede asumir que el script se ejecuta sobre una base de datos vacía.

Eliminación y modificación de triggers

Para eliminar un trigger:

```

DROP TRIGGER IF EXISTS trg_nuevo_pokemon;

```

Para modificar un trigger, hay que eliminarlo y volver a crearlo con la nueva definición.

Comportamiento con las claves foráneas

El trigger **trg_nuevo_pokemon** inserta en **log_nuevos** una referencia a **pokemon(id)**. Como la tabla **log_nuevos** tiene una clave foránea hacia **pokemon**, si más tarde se borra un Pokémon, la fila de auditoría

correspondiente se eliminará gracias al **ON DELETE CASCADE**. Así se conserva la integridad referencial sin esfuerzo adicional.

Cuidado con la recursividad

Un trigger puede provocar que se ejecuten otros triggers (porque su acción afecta a otra tabla que también tiene triggers). SQLite limita la profundidad de la recursión para evitar bucles infinitos, pero hay que diseñarlos con cuidado. En nuestro caso, **trg_nuevo_pokemon** solo inserta en **log_nuevos**, que no tiene triggers que afecten a **pokemon**, así que no hay riesgo.

Relación con el proyecto sin internet

Los triggers ejecutan su código dentro del motor de SQLite, sin depender de una red ni de una aplicación externa. Si un compañero comparte la base de datos **pokemon.db** y alguien inserta un Pokémon desde la terminal, el log se actualizará. Si posteriormente se consulta la tabla **log_nuevos**, se podrá ver el historial de incorporaciones, incluso si la inserción ocurrió cuando no había conexión a internet (porque la inserción se hizo localmente).

Esta automatización refuerza la idea de que la base de datos no es un simple almacén pasivo, sino un componente activo que puede encargarse de tareas de mantenimiento de datos. Con los triggers, nuestra Pokédex local gana en robustez y trazabilidad.

Conclusión

Los triggers son una herramienta poderosa para encapsular lógica relacionada con eventos de datos. En nuestra Pokédex hemos implementado un caso clásico: auditoría de inserciones. Los estudiantes pueden ahora experimentar añadiendo triggers para otros eventos, por ejemplo, un trigger que registre cuándo se modifica la altura de un Pokémon, o que impida borrar un tipo si todavía tiene Pokémon asociados. En el próximo tema abordaremos el concepto de procedimientos almacenados en el contexto de SQLite, donde explicaremos por qué no existen como tales y cómo suplirlos con funciones de Python o combinaciones de triggers.

Tema 10: Procedimientos almacenados en SQLite

Cuando trabajamos con sistemas como MySQL, PostgreSQL u Oracle, es habitual encapsular lógica compleja en **procedimientos almacenados**: bloques de código SQL que residen en la propia base de datos y que pueden invocarse desde la aplicación con una simple llamada. Sin embargo, SQLite **no incorpora este concepto**. En este tema veremos por qué SQLite tomó esa decisión de diseño, qué alternativas nos ofrece para conseguir el mismo efecto y cómo aplicarlas en nuestro proyecto de la Pokédex local para mantener el código organizado y reutilizable incluso sin conexión a internet.

¿Qué es un procedimiento almacenado?

Un procedimiento almacenado es una rutina que contiene una o varias sentencias SQL, junto con lógica de control (variables, condicionales, bucles) y que se almacena en el servidor de base de datos. Se puede ejecutar con un simple **CALL nombre_procedimiento(parámetros)**. Sus ventajas típicas son:

- Centralizar lógica de negocio en la base de datos.
- Reducir el tráfico de red (la aplicación solo invoca, no envía múltiples consultas).
- Seguridad, al poder conceder permisos de ejecución sin dar acceso directo a las tablas.

Sin embargo, SQLite no sigue el modelo cliente-servidor; es una biblioteca embebida que corre en el mismo proceso de la aplicación. Por tanto, no existe un servidor separado donde almacenar esa lógica.

La filosofía de SQLite: sin procedimientos almacenados

Los desarrolladores de SQLite decidieron no incluir procedimientos almacenados por varias razones:

1. **Simplicidad:** mantener el motor pequeño y fácil de integrar. Añadir un lenguaje de procedimientos similar a PL/SQL o PL/pgSQL aumentaría enormemente la complejidad y el tamaño de la biblioteca.
2. **Modelo embebido:** la aplicación y la base de datos comparten el mismo espacio de memoria y procesamiento. No hay ventaja en ejecutar lógica "dentro" del motor si la aplicación ya la puede ejecutar en su propio lenguaje, que además es más expresivo.
3. **Portabilidad:** al no tener un lenguaje propietario, los archivos `.db` son totalmente portables entre sistemas sin depender de versiones o dialectos de procedimientos.

Esto no significa que estemos limitados; podemos trasladar la lógica al lado de la aplicación (Python, en nuestro caso) y también contamos con otras herramientas nativas de SQLite: **triggers**, **vistas** y **funciones definidas por el usuario** (UDF).

Alternativa 1: Funciones en la aplicación (Python)

Esta es la solución más directa y la que ya hemos aplicado en nuestro proyecto. En lugar de un procedimiento almacenado, escribimos una función Python que ejecuta varias sentencias SQL de manera coordinada. Por ejemplo, la inserción completa de un Pokémon desde los datos de la API (insertar tipos, insertar Pokémon, insertar relaciones) la encapsulamos en una función:

```
def insertar_pokemon(conexion, poke_data):
    """Inserta un Pokémon y sus tipos en la base de datos."""
    cursor = conexion.cursor()
    poke_id = poke_data["id"]
    nombre = poke_data["name"]
    altura = poke_data["height"]
    peso = poke_data["weight"]
    experiencia = poke_data["base_experience"]
    tipos = [t["type"]["name"] for t in poke_data["types"]]

    # 1. Insertar cada tipo (si no existe)
    for tipo_nombre in tipos:
        cursor.execute(
            "INSERT OR IGNORE INTO tipos (nombre) VALUES (?)",
            (tipo_nombre,)
        )

    # 2. Insertar el Pokémon
    cursor.execute(
```

```

        "INSERT INTO pokemon (id, nombre, altura, peso, experiencia_base)
VALUES (?, ?, ?, ?, ?)",
        (poke_id, nombre, altura, peso, experiencia)
    )

    # 3. Insertar relaciones
    for tipo_nombre in tipos:
        cursor.execute("SELECT id FROM tipos WHERE nombre = ?", (tipo_nombre,))
        tipo_id = cursor.fetchone()[0]
        cursor.execute(
            "INSERT OR IGNORE INTO pokemon_tipo (pokemon_id, tipo_id) VALUES
            (?, ?)",
            (poke_id, tipo_id)
        )
    # Nota: el commit se hace fuera, o podemos hacerlo aquí si pasamos la
    conexión

```

Ahora, cada vez que necesitemos guardar un Pokémon, simplemente llamamos a `insertar_pokemon(conexion, datos)`. Esto es exactamente el equivalente a un procedimiento almacenado, pero escrito en Python y ejecutándose en el mismo proceso que la base de datos. No genera tráfico de red extra porque todo es local.

La ventaja es que podemos aprovechar todas las capacidades de Python (depuración, bibliotecas externas, manejo de errores con try/except) y mantener el código junto al resto de la aplicación. En nuestro flujo de trabajo, esta función puede ser llamada desde un bucle que recorre la lista de Pokémon descargada de la API, justo antes de desconectarnos de internet.

Alternativa 2: Triggers para automatizaciones simples

Los triggers que vimos en el tema 9 pueden verse como una forma primitiva de procedimiento almacenado: se disparan automáticamente ante eventos de datos. Son útiles para auditoría, validación o mantenimiento de integridad. Sin embargo, no pueden reemplazar la lógica compleja porque:

- Solo se ejecutan en respuesta a `INSERT`, `UPDATE` o `DELETE` sobre una tabla concreta.
- No admiten parámetros de entrada arbitrarios (solo pueden acceder a `NEW` y `OLD`).
- No pueden ser invocados manualmente con `CALL`.

En nuestro proyecto, `trg_nuevo_pokemon` automatiza el registro de nuevos Pokémon, pero la lógica principal de inserción sigue estando en Python.

Alternativa 3: Funciones definidas por el usuario (UDF)

SQLite permite extender el lenguaje SQL con funciones escritas en el lenguaje anfitrión (Python, C, etc.) mediante la API `create_function()`. Estas funciones pueden usarse dentro de cualquier consulta SQL, aportando una potencia similar a los procedimientos, aunque no realicen múltiples sentencias por sí solas.

Por ejemplo, podríamos definir una función que convierta la altura de decímetros a metros, para mostrarla en consultas de manera más amigable:

```
import sqlite3

def dm_a_metros(dm):
    return dm / 10.0

conexion = sqlite3.connect("pokemon.db")
conexion.create_function("metros", 1, dm_a_metros)

# Ahora podemos usarla en SQL:
cursor = conexion.cursor()
cursor.execute("SELECT nombre, altura, metros(altura) FROM pokemon")
for fila in cursor.fetchall():
    print(f"Pokémon: {fila[0]}, Altura: {fila[2]} m")
```

La función `metros` está disponible durante toda la vida de la conexión. Podríamos hacer algo más complejo, como una función que devuelva la suma de las estadísticas de un Pokémon, aunque eso implicaría consultas internas que es mejor manejar en Python para no complicar el código SQL.

Las UDF son una herramienta intermedia: permiten inyectar lógica personalizada en las consultas, pero no reemplazan un procedimiento con múltiples pasos.

Alternativa 4: Scripts SQL completos

Otra forma de empaquetar lógica es mediante un archivo `.sql` que contenga todas las sentencias necesarias y ejecutarlo con `executescript()`. Ya lo usamos con `esquema.sql` para crear las tablas. Podríamos tener un script `cargar_pokemon.sql` que tome literales, pero no admitiría parámetros variables de forma nativa. Por tanto, no es práctico para inserciones dinámicas.

Comparativa de alternativas

Método	Complejidad soportada	Reutilización	Flexibilidad
Función Python	Alta (cualquier lógica)	Muy alta	Total
Triggers	Baja/media	Alta (automática)	Limitada a eventos
UDF (<code>create_function</code>)	Media (cálculos, transformaciones)	Alta dentro de SQL	Media
Script SQL	Solo SQL estático	Media	Baja (sin parámetros)

Para nuestro objetivo —insertar datos descargados de la PokéAPI— la función Python es la opción más clara y didáctica.

Aplicación en la Pokédex local

Recordemos el contexto: tenemos un script que se conecta a internet, descarga los datos de la PokéAPI y los inserta en la base de datos local. Ese script utilizará la función `insertar_pokemon()` definida

anteriormente. Una vez finalizada la descarga, nos desconectamos de la red y solo usamos consultas SQL (con ayuda de la vista `vista_pokemon_tipos`) para explorar la información. En ningún momento necesitamos procedimientos almacenados en el motor, porque la lógica de carga ya se ejecutó en Python.

Si más tarde quisiéramos añadir más Pokémon manualmente (sin API), podríamos escribir un pequeño programa que recoja los datos por teclado y llame a la misma función, manteniendo el código organizado.

Conclusión

SQLite no ofrece procedimientos almacenados nativos porque su diseño embebido hace innecesaria la separación entre lógica de aplicación y motor. La mejor práctica es implementar la lógica de negocio en el lenguaje de la aplicación (Python), apoyándose en triggers para automatizaciones internas y en UDF para cálculos puntuales. En nuestro proyecto, esto se traduce en funciones Python que ejecutan las transacciones de inserción de manera limpia, manteniendo la base de datos simple y portable. Con esto completamos la fase de almacenamiento; los siguientes pasos son las consultas avanzadas y la explotación de la información ya persistida localmente.

Tema 11: Consultas y extracción de datos locales

Una vez que la base de datos ha sido poblada con los Pokémon descargados de la PokéAPI, dejamos de depender de la conexión a internet. El verdadero valor de nuestro trabajo aparece ahora: podemos extraer información, hacer búsquedas, generar listados y analizar los datos usando únicamente SQL. En este tema repasaremos las consultas fundamentales (SELECT, JOIN, WHERE, GROUP BY, ORDER BY) aplicadas a nuestro esquema de Pokédex, y veremos cómo la vista que definimos nos simplifica las operaciones más habituales.

Recuperar todos los Pokémon y sus tipos

La consulta más directa es listar cada Pokémon con los nombres de sus tipos. En lugar de repetir la unión de tres tablas cada vez, usaremos la vista `vista_pokemon_tipos` que ya creamos:

```
SELECT * FROM vista_pokemon_tipos;
```

Esta vista devuelve columnas `id`, `nombre` y `tipos` (una cadena con los tipos separados por comas). Como es una tabla virtual, podemos aplicarle cualquier cláusula adicional, como `ORDER BY`:

```
SELECT * FROM vista_pokemon_tipos ORDER BY nombre;
```

Desde Python también podríamos ejecutar esa misma consulta y procesar los resultados:

```
cursor.execute("SELECT * FROM vista_pokemon_tipos ORDER BY nombre")
for fila in cursor.fetchall():
    print(f"{fila[0]}: {fila[1]} - Tipos: {fila[2]}")
```

Búsquedas con WHERE

Podemos filtrar por campos de las tablas base aunque estemos usando la vista. Por ejemplo, para buscar un Pokémon por nombre (exacto o parcial con LIKE):

```
SELECT * FROM vista_pokemon_tipos WHERE nombre = 'pikachu';
```

Búsqueda por coincidencia de patrón (todos los que empiecen con "char"):

```
SELECT * FROM vista_pokemon_tipos WHERE nombre LIKE 'char%';
```

Si necesitamos buscar por tipo, podemos filtrar directamente sobre la columna **tipos** de la vista. Al ser una cadena que contiene los tipos concatenados, podemos usar **LIKE**:

```
SELECT * FROM vista_pokemon_tipos WHERE tipos LIKE '%fuego%';
```

Esta es una manera rápida, aunque para búsquedas más estrictas podríamos volver a las tablas base y usar **JOIN**:

```
SELECT p.nombre
FROM pokemon p
JOIN pokemon_tipo pt ON p.id = pt.pokemon_id
JOIN tipos t ON pt.tipo_id = t.id
WHERE t.nombre = 'water';
```

Así obtenemos exactamente los Pokémon que tienen el tipo agua, sin falsos positivos que podrían darse si un nombre contuviera la palabra "water" en otro contexto. Sin embargo, para los nombres de tipos reales no hay ambigüedad, por lo que usar la vista suele ser suficiente.

Uniones manuales (JOIN)

Aunque la vista nos oculta la complejidad, es necesario entender cómo se construyen las uniones, porque en algún momento necesitaremos hacer consultas que la vista no cubra. La base de cualquier unión en nuestro esquema es:

- Partimos de **pokemon**.

- Unimos con `pokemon_tipo` mediante `pokemon.id = pokemon_tipo.pokemon_id`.
- Unimos con `tipos` mediante `pokemon_tipo.tipo_id = tipos.id`.

Si queremos obtener, por ejemplo, el nombre del Pokémon, su altura y todos los tipos en una columna concatenada:

```
SELECT p.nombre, p.altura, GROUP_CONCAT(t.nombre, ', ') AS tipos
FROM pokemon p
JOIN pokemon_tipo pt ON p.id = pt.pokemon_id
JOIN tipos t ON pt.tipo_id = t.id
GROUP BY p.id;
```

Nótese el uso de `GROUP BY` y `GROUP_CONCAT`. Si solo hiciéramos la unión sin agrupar, cada Pokémon aparecería tantas veces como tipos tenga (una fila por cada tipo). Con `GROUP BY p.id` y `GROUP_CONCAT` juntamos toda la información en una sola fila.

Ordenación (ORDER BY)

Podemos ordenar los resultados por cualquier columna. Para listar los Pokémon del más pesado al más ligero:

```
SELECT nombre, peso FROM pokemon ORDER BY peso DESC;
```

O ordenar por nombre y, en caso de empate, por experiencia base:

```
SELECT nombre, experiencia_base FROM pokemon ORDER BY nombre, experiencia_base;
```

La ordenación también funciona sobre columnas calculadas o sobre los resultados de una vista:

```
SELECT * FROM vista_pokemon_tipos ORDER BY tipos, nombre;
```

Funciones de agregación

Con los datos locales podemos realizar estadísticas. Por ejemplo, contar cuántos Pokémon tenemos:

```
SELECT COUNT(*) AS total_pokemon FROM pokemon;
```

Calcular la altura media de todos los Pokémon:

```
SELECT AVG(altura) AS altura_media FROM pokemon;
```

Encontrar el Pokémon más pesado:

```
SELECT nombre, peso FROM pokemon ORDER BY peso DESC LIMIT 1;
```

Para contar cuántos Pokémon hay de cada tipo, debemos usar la tabla intermedia:

```
SELECT t.nombre AS tipo, COUNT(pt.pokemon_id) AS cantidad
FROM tipos t
LEFT JOIN pokemon_tipo pt ON t.id = pt.tipo_id
GROUP BY t.id
ORDER BY cantidad DESC;
```

Esta consulta lista todos los tipos y la cantidad de Pokémon que poseen, incluso aquellos tipos que no tienen Pokémon (aparecerían con 0). Si un Pokémon tiene dos tipos, se contabiliza en ambos.

Subconsultas

Podemos usar subconsultas para responder preguntas más complejas. Por ejemplo, obtener los Pokémon que pesan más que el peso promedio:

```
SELECT nombre, peso
FROM pokemon
WHERE peso > (SELECT AVG(peso) FROM pokemon);
```

O buscar los Pokémon que comparten al menos un tipo con Pikachu:

```
SELECT DISTINCT p.nombre
FROM pokemon p
JOIN pokemon_tipo pt ON p.id = pt.pokemon_id
WHERE pt.tipo_id IN (
    SELECT pt2.tipo_id
    FROM pokemon_tipo pt2
    JOIN pokemon p2 ON pt2.pokemon_id = p2.id
    WHERE p2.nombre = 'pikachu'
)
AND p.nombre <> 'pikachu';
```

Consultas desde Python sin conexión

La ventaja clave del proyecto es que todas estas consultas se ejecutan localmente. Un script Python de consulta no necesita internet:

```
import sqlite3

conexion = sqlite3.connect("pokemon.db")
conexion.execute("PRAGMA foreign_keys = ON")
cursor = conexion.cursor()

# Ejemplo: pedir al usuario un tipo y listar Pokémon
tipo = input("Introduce un tipo de Pokémon: ").lower()
cursor.execute(
    "SELECT * FROM vista_pokemon_tipos WHERE LOWER(tipos) LIKE ?",
    (f"%{tipo}%",)
)
resultados = cursor.fetchall()

if resultados:
    print(f"Pokémon de tipo {tipo}:")
    for fila in resultados:
        print(f"  - {fila[1]} (ID: {fila[0]})")
else:
    print("No se encontraron Pokémon de ese tipo.")
conexion.close()
```

Este pequeño programa funciona completamente offline después de haber poblado la base de datos con los datos de la API. Así se cumple el objetivo inicial: superar el problema de la conexión a internet.

Optimización de las consultas

Gracias a los índices que creamos en el tema 7, todas estas consultas se ejecutarán de forma eficiente. La búsqueda por nombre (`WHERE nombre = ...`) usa `idx_pokemon_nombre`. La búsqueda por tipo (cuando hacemos `JOIN` con `pokemon_tipo` filtrando por `tipo_id`) usa `idx_pokemon_tipo_tipo`. Incluso la vista se beneficia de esos índices porque internamente el motor optimiza las uniones.

Si realizamos muchas consultas sobre el peso o la altura, podríamos plantearnos crear índices adicionales:

```
CREATE INDEX idx_pokemon_peso ON pokemon(peso);
```

Pero como la tabla `pokemon` rara vez superará unos cientos de registros, no es imprescindible. Es útil mencionarlo para que los estudiantes entiendan que los índices se crean en función de las consultas reales.

Exportación de resultados

Una vez que tenemos los datos locales, podemos llevar los resultados de una consulta a un archivo CSV para tratarlos con otras herramientas. Desde la línea de comandos de SQLite es directo:

```
sqlite3 pokemon.db ".headers on" ".mode csv" "SELECT * FROM
vista_pokemon_tipos" > pokedex.csv
```

Desde Python podemos hacer lo mismo con el módulo `csv`:

```
import csv
cursor.execute("SELECT * FROM vista_pokemon_tipos")
with open("pokedex.csv", "w", newline="", encoding="utf-8") as f:
    escritor = csv.writer(f)
    escritor.writerow(["ID", "Nombre", "Tipos"])
    escritor.writerows(cursor.fetchall())
```

Así, incluso sin internet, podemos extraer subconjuntos de la Pokédex local a formatos intercambiables.

Conclusión

Las consultas SQL son la herramienta final que cierra el ciclo del proyecto. Hemos pasado de una dependencia total de la PokéAPI (online) a disponer de un sistema de almacenamiento local con el que podemos:

- Buscar Pokémon por nombre, tipo, peso, altura.
- Obtener estadísticas agregadas.
- Generar listados ordenados.
- Exportar datos.

Todo ello sin necesidad de volver a conectarnos a la red. Los estudiantes pueden ampliar estas consultas según sus intereses: buscar el Pokémon más alto de un tipo, los que dan más experiencia, etc. Nuestra Pokédex local ya es una herramienta autónoma y eficiente, construida pieza a pieza sobre SQLite.

Preguntas y ejercicios de práctica

Esta sección reúne preguntas de repaso y ejercicios prácticos que abarcan todos los temas trabajados en el curso. Están pensados para que los estudiantes consoliden lo aprendido sobre SQLite y su aplicación al proyecto de la Pokédex local, reforzando tanto los conceptos teóricos como la capacidad de escribir y probar código por sí mismos.

Preguntas de repaso

Tema 1 – Introducción a SQLite

1. ¿Qué significa que SQLite sea una base de datos embebida y sin servidor? ¿Qué ventajas aporta en nuestro proyecto escolar?
2. Menciona tres dispositivos o aplicaciones cotidianas que usan SQLite.

3. ¿Por qué un solo archivo `.db` resulta útil cuando queremos desconectarnos de internet después de descargar los datos de la PokéAPI?
4. ¿Cuál es la principal limitación de SQLite respecto a la concurrencia? ¿Afecta a nuestro uso en el aula?

Tema 2 – Instalación y herramientas

5. ¿Qué dos herramientas básicas necesitamos como mínimo para trabajar con SQLite? ¿En qué situaciones usarías cada una?
6. ¿Cómo podemos comprobar desde la terminal y desde Python que SQLite está correctamente disponible?
7. ¿Para qué sirve el comando `.tables` dentro del shell interactivo de `sqlite3`? ¿Y `.schema`?
8. ¿Qué significa que el módulo `sqlite3` pertenezca a la biblioteca estándar de Python? ¿Por qué eso facilita el trabajo en el aula?

Tema 3 – Crear y conectarse a una base de datos

9. ¿Qué sucede si intentamos conectarnos con `sqlite3.connect("pokemon.db")` y el archivo no existe?
10. ¿Por qué es obligatorio ejecutar `PRAGMA foreign_keys = ON` después de abrir una conexión? Explica qué ocurriría si no lo hiciéramos.
11. ¿Cuál es la diferencia entre conectarse a un archivo `.db` y a `:memory:`? ¿Cuándo conviene cada uno?

Tema 4 – Tipos de datos y creación de tablas

12. ¿Qué es la afinidad de tipos en SQLite? Pon un ejemplo con una columna declarada como `VARCHAR(100)`.
13. ¿Por qué en la tabla `tipos` usamos `AUTOINCREMENT` y en `pokemon` no?
14. ¿Qué restricción impide que insertemos dos tipos con el mismo nombre? ¿Dónde se define?
15. ¿Qué diferencia hay entre `PRIMARY KEY` simple y compuesta? Ilustra con la tabla `pokemon_tipo`.

Tema 5 – Modelado de relaciones con el dominio Pokémon

16. ¿Por qué no podemos guardar los tipos de un Pokémon directamente en una columna de la tabla `pokemon`?
17. ¿Qué es una tabla intermedia (o de enlace) y qué problema resuelve?
18. ¿Qué significan las cláusulas `ON DELETE CASCADE` en nuestras claves foráneas? Da un ejemplo concreto con un Pokémon y sus tipos.
19. ¿En qué orden debemos insertar los datos si queremos respetar las claves foráneas? ¿Por qué?

Tema 6 – Inserción de datos desde la API

20. ¿Por qué usamos `INSERT OR IGNORE` en la tabla `tipos`? ¿Qué ocurriría si usáramos solo `INSERT`?
21. Explica la diferencia entre las sentencias parametrizadas y la concatenación de cadenas en SQL. ¿Por qué es importante usar parámetros?
22. Si durante la inserción falla la conexión a la PokéAPI después de haber insertado 80 de 151 Pokémon, ¿cómo deberíamos actuar para no perder los datos ya guardados?

Tema 7 – Índices

23. ¿Qué es un índice y por qué mejora el rendimiento de las consultas?
24. ¿Por qué SQLite crea automáticamente un índice para la clave primaria y para las columnas `UNIQUE`?
25. ¿Por qué creamos un índice adicional sobre `tipo_id` en la tabla `pokemon_tipo`? ¿Qué consulta concreta se beneficia de él?
26. ¿Cuál es el principal inconveniente de tener demasiados índices? ¿Es un problema grave en nuestro proyecto?

Tema 8 – Vistas

27. ¿Qué es una vista y en qué se diferencia de una tabla normal?
28. Explica con tus propias palabras qué hace la vista `vista_pokemon_tipos`. ¿Qué ocurre con los datos de la vista si luego añadimos un nuevo Pokémon?
29. ¿Podemos hacer un `INSERT` sobre `vista_pokemon_tipos`? Justifica la respuesta.
30. ¿Qué ventaja nos da usar la vista en lugar de escribir la consulta con `JOIN` cada vez?

Tema 9 – Triggers

31. ¿Qué es un trigger y cuándo se ejecuta?
32. ¿Qué significan las referencias `NEW` y `OLD` dentro de un trigger? Pon un ejemplo con nuestro trigger `trg_nuevo_pokemon`.
33. ¿Por qué el trigger de auditoría es `AFTER INSERT` y no `BEFORE INSERT`?
34. ¿Se podría crear un trigger que impida borrar un tipo si todavía hay Pokémon que lo usan? ¿Cómo lo harías?

Tema 10 – Procedimientos almacenados

35. ¿Por qué SQLite no incorpora procedimientos almacenados como otros sistemas?
36. ¿Cuál es la alternativa recomendada en nuestro proyecto para encapsular la lógica de inserción de un Pokémon?
37. ¿Qué son las funciones definidas por el usuario (UDF) y qué limitaciones tienen respecto a los procedimientos almacenados tradicionales?

Tema 11 – Consultas y extracción de datos locales

38. Escribe una consulta SQL que devuelva el nombre y el peso de los Pokémon cuyo peso supere los 100 hectogramos, ordenados de mayor a menor peso.
39. ¿Para qué sirve la función `GROUP_CONCAT`? ¿En qué vista la hemos utilizado?
40. ¿Cómo podemos exportar los resultados de una consulta a un archivo CSV desde la terminal de SQLite?

Ejercicios prácticos

Ejercicio 1: Instalación y primera conexión

Asegúrate de tener disponible `sqlite3` en la terminal y el módulo `sqlite3` en Python. Crea una base de datos llamada `pokemon.db` y, desde la terminal, muestra la lista de tablas con `.tables`. Luego, desde

Python, conéctate a la misma base de datos, activa la restricción de claves foráneas e imprime la versión de SQLite.

```
# Escribe aquí tu solución
import sqlite3
# ... tu código ...
```

Ejercicio 2: Creación del esquema completo

Basándote en los temas 4 y 5, crea un archivo `esquema.sql` con todas las instrucciones necesarias para dejar la base de datos lista para recibir los datos de la PokéAPI. Debe incluir las tablas `tipos`, `pokemon`, `pokemon_tipo`, la tabla de auditoría `log_nuevos`, los índices explicados, la vista `vista_pokemon_tipos` y el trigger `trg_nuevo_pokemon`. Después, desde Python, ejecuta ese script sobre `pokemon.db`.

```
# Tu código para ejecutar el script
```

Ejercicio 3: Carga de datos desde la API

Escribe un programa en Python que se conecte a la PokéAPI (puedes usar la URL <https://pokeapi.co/api/v2/pokemon/> con números del 1 al 151) y llene la base de datos con todos los Pokémon de la primera generación. Implementa una función `guardar_pokemon(conexion, datos_json)` que realice las inserciones necesarias respetando el orden lógico.

```
import requests
import sqlite3

def guardar_pokemon(conexion, datos_json):
    # Completa el código
    pass

# Carga de los 151 Pokémon
```

Ejercicio 4: Consulta de la Pokédex local

Una vez poblada la base de datos, escribe un programa Python que ofrezca un pequeño menú interactivo (offline) con las siguientes opciones:

1. Buscar Pokémon por nombre.
2. Listar todos los Pokémon de un tipo dado.
3. Mostrar los 10 Pokémon más pesados.
4. Mostrar la altura media de todos los Pokémon.
5. Exportar toda la Pokédex a un archivo CSV.

Utiliza la vista y las tablas según convenga.

Ejercicio 5: Verificación de índices

Ejecuta `EXPLAIN QUERY PLAN` para las siguientes consultas antes y después de eliminar el índice `idx_pokemon_nombre`. Comprueba el plan de ejecución para entender la diferencia.

- `SELECT * FROM pokemon WHERE nombre = 'charizard';`
- `SELECT * FROM pokemon WHERE nombre LIKE 'char%';`

```
-- Ejemplo de consulta con EXPLAIN
EXPLAIN QUERY PLAN SELECT * FROM pokemon WHERE nombre = 'charizard';
```

Ejercicio 6: Ampliación del modelo (opcional)

Investiga cómo añadir una tabla de movimientos (`movimientos`) y relacionarla con los Pokémon de forma muchos a muchos. Crea las tablas y los índices necesarios, modifica la vista para que muestre también los movimientos concatenados e inserta algunos datos de ejemplo. Justifica las decisiones de diseño.

Ejercicio 7: Trigger de control

Modifica el trigger `trg_nuevo_pokemon` para que además de registrar el `pokemon_id`, inserte también el nombre del Pokémon en el log. (Pista: usa `NEW.nombre`). Prueba que el nuevo trigger funciona insertando un Pokémon de prueba.

```
-- Tu nueva definición del trigger
```

Ejercicio 8: Función definida por el usuario

Crea una función Python que convierta el peso de hectogramos a kilogramos y regístrala con `create_function` para que puedas usarla en consultas como `SELECT nombre, peso_kg(peso) FROM pokemon`.

```
def peso_kg(hectogramos):
    # ...
    pass

conexion.create_function("peso_kg", 1, peso_kg)
```

Ejercicio 9: Depuración de errores comunes

Tu compañero ha ejecutado el script de carga varias veces y ahora tiene Pokémon duplicados en la tabla `pokemon` (mismo ID repetido). ¿Qué sentencia usó incorrectamente? ¿Cómo puede limpiar la base de datos sin volver a crearla desde cero? Diseña una estrategia de reparación.

Ejercicio 10: Reflexión final

Escribe un breve párrafo explicando cómo este proyecto de Pokédex local te ha ayudado a entender las bases de datos relacionales y a resolver el problema de la dependencia de internet. Menciona al menos tres conceptos de SQLite que hayas aplicado directamente en la solución.

Estas preguntas y ejercicios están diseñados para que los estudiantes trabajen de manera progresiva, desde la verificación de la instalación hasta la extensión del modelo. El docente puede seleccionar aquellos que mejor se adapten al ritmo del grupo o utilizarlos como base para una evaluación final del módulo de bases de datos con SQLite.